TECHNICAL WHITEPAPER

Constrained Certificate Trust in Chrome

Exclusive Domain-to-CA Binding for ***.scone.cloud**

Question addressed

Can Chrome be forced to accept a protected domain only when the verified certificate chain contains an explicitly approved CA?

Architecture analysis • policy configuration • operational runbooks • Chromium enhancement proposal

Version 2.1 | 26 July 2026

Sources verified against current Chrome Enterprise documentation and Chromium main.

Document control

Table 1 — Document status and scope

Field	Value
Title	Constrained Certificate Trust in Chrome
Subtitle	Exclusive Domain-to-CA Binding for *.scone.cloud
Document type	Technical whitepaper and operational runbook
Version	2.1
Publication date	26 July 2026
Source baseline	Chrome Enterprise policy documentation and Chromium main, verified 26 July 2026
Primary conclusion	Stock Chrome has no domain-scoped exclusive trust-anchor allowlist. A single-purpose TCP-only profile can approximate exclusivity by globally distrusting every other active anchor, but the design is global, brittle and requires continuous root-store reconciliation.
Revision 2.1 focus	Analysis and runbook for removing or globally distrusting public roots, re-adding roots with namespace exclusions, distrust precedence, Chrome Root Store updates and QUIC.

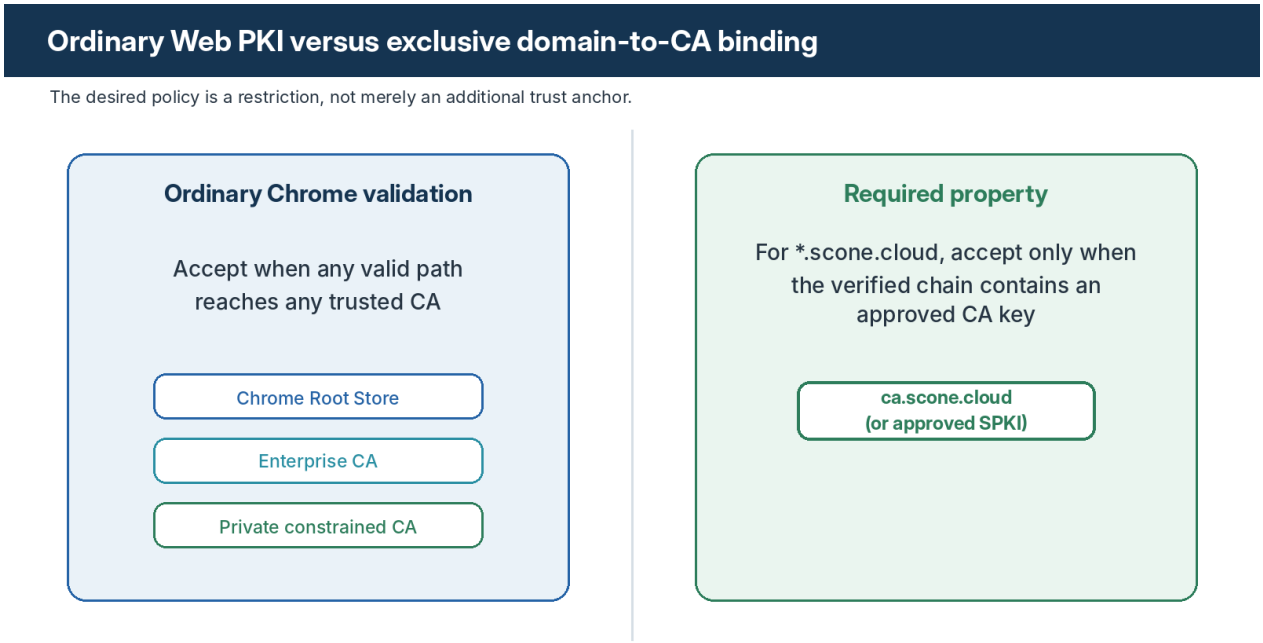
Accuracy note Chrome policy names and Chromium implementation details are current as of the publication date. Enterprise administrators should validate policy availability against the browser channel deployed in their organization before rollout.

Executive summary

This whitepaper evaluates a precise security requirement: when Chrome connects to a service whose reference hostname is within the DNS subtree scone.cloud, the browser should accept the TLS connection only when the validated certificate chain contains a CA certificate or CA public key explicitly approved by the organization. A representative protected host is api.scone.cloud, and the designated CA is referred to as ca.scone.cloud. The CA label is descriptive; the operational identity must be a verified certificate fingerprint or SubjectPublicKeyInfo (SPKI) hash.

Chrome 132 and later provides the CACertificatesWithConstraints enterprise policy. It can add a CA trust anchor while attaching permitted DNS or IP constraints outside the certificate. Chrome also provides CACertificates, which adds enterprise trust anchors and, in current Chromium, enforces embedded anchor constraints and the anchor validity interval. These mechanisms are valuable because they prevent a private CA from being used outside its intended namespace. They do not, however, reserve that namespace for the private CA. [3][4][5]

The decisive limitation is that the Chrome certificate verifier normally accepts a connection when it can construct at least one valid certification path to any eligible trust anchor. Adding ca.scone.cloud with a constraint for scone.cloud creates another eligible path. It does not remove public roots from the Chrome Root Store, and it does not tell Chrome to reject a valid chain from another trusted CA. A valid public-CA certificate for api.scone.cloud therefore remains acceptable unless a separate control removes or filters that trust path. [1][2][3]



A qualified single-purpose workaround exists. If every public and enterprise trust-anchor SPKI other than ca.scone.cloud is globally distrusted, platform integration is disabled, the approved CA is added with a permitted subtree of scone.cloud, and the connection uses TCP-based TLS, the effective anchor set can be reduced to the approved CA. In that profile, a public-CA certificate for api.scone.cloud fails. This is not a domain-scoped Chrome feature: ordinary public HTTPS also fails, current Chrome provides no supported policy that replaces the Chrome Root Store with an empty store, and the distrust list must be reconciled after every root-store or browser update. [5][7][11][23][24][25][26]

The alternative idea—re-add every ordinary public root but exclude scone.cloud from each root—would preserve general browsing in principle. Stock Chrome cannot express it. CACertificatesWithConstraints exposes permitted_dns_names and permitted_cidrs, not excluded_dns_names, and a certificate SPKI placed in CADistrustedCertificates remains distrusted even when the same key is also added through a trust policy. [1][5][7][24][25]

Figure 1 — Ordinary Web PKI accepts any valid trust path; exclusive binding requires a namespace-specific restriction.

Table 2 — Key findings

Question	Answer	Operational implication

Question	Answer	Operational implication
Can Chrome add ca.scone.cloud only for scone.cloud?	Yes. Use CACertificatesWithConstraints with permitted_dns_names=["scone.cloud"], or use a CA certificate with a critical embedded Name Constraints extension.	This limits how the added CA can be used.
Can stock Chrome require ca.scone.cloud for every *.scone.cloud connection?	No domain-scoped policy currently expresses that requirement.	A public or other eligible enterprise CA can still validate the same hostname.
Can CAPlatformIntegrationEnabled=false solve the problem?	Only partially. It suppresses user-added platform trust anchors when Chrome Root Store verification is used; it does not disable the Chrome Root Store.	Useful endpoint hardening, not exclusive binding.
Can CADistrustedCertificates be used?	It can globally distrust listed SPKIs, not only for scone.cloud.	A dedicated kiosk profile could distrust almost everything, but the design is brittle and fail-open when the root store changes.
What provides the exact guarantee?	Either a custom Chromium verifier that filters trust anchors by hostname, or a mandatory non-bypassable TLS policy proxy that performs that check upstream.	Choose browser-local enforcement for highest assurance; choose a proxy for easier enterprise deployment.
Can a dedicated profile effectively trust only ca.scone.cloud?	Conditionally yes for TCP-based TLS: globally distrust every other active root/enterprise SPKI, disable platform integration, add ca.scone.cloud with the scone.cloud constraint and disable QUIC.	This is a single-purpose-browser fallback. General public HTTPS fails and the root-store denylist must remain complete after updates.
Can public roots be re-added with an exclusion for scone.cloud?	Not with current stock-Chrome policy. External constraints have no excluded_dns_names field, and global distrust overrides re-trust of the same SPKI.	Preserving the public Web while reserving scone.cloud still requires hostname-scoped anchor filtering or a non-bypassable proxy.

> Recommended design Deploy constrained trust and endpoint hardening as the baseline, then add exactly one exclusive enforcement point: custom Chromium anchor filtering or a mandatory TLS policy proxy backed by egress controls. A complete global-distrust profile is a single-purpose fallback only. Publish CAA and monitor certificate transparency as defense in depth, not as substitutes for browser enforcement.

How to read this whitepaper

- Chapters 1–6 establish the security property and explain why stock Chrome cannot provide it exclusively.
- Chapters 7–8 compare architectures and present the recommended layered design.
- Chapter 9 is an operational runbook for CA preparation, Chrome policy deployment, testing, proxy enforcement, browser modification, rotation and incident response.
- Chapters 10–12 provide a security analysis, a concrete Chromium policy proposal and integration with confidential-computing services.
- The appendices contain copy-ready policy, OpenSSL, PowerShell and macOS configuration examples, plus a dedicated-browser root-store reconciliation runbook.

Contents

Document control 2
Executive summary 3
1. Problem statement and security objective 6
10. Security analysis 35
11. Chromium enhancement proposal: RequiredTrustAnchors 37
12. Applying the design to confidential-computing services 39

Document control 2
2. Chrome certificate verification architecture 8
3. Name Constraints and constrained certificate trust 10
4. Chrome enterprise policies relevant to certificate trust 12
5. Case study: ca.scone.cloud for the scone.cloud namespace 14
6. Can stock Chrome enforce that only the explicit CA is used? 16
7. Evaluation of alternative mechanisms 21
8. Recommended architecture 23
9. Operational runbook 25
10. Security analysis 35
13. Conclusions and recommendations 41
Appendix A. Complete Linux baseline policy 42
Appendix B. Embedded Name Constraints checklist 43
Appendix C. Exclusive-enforcement test matrix 44
Appendix D. Evidence collection commands 46
Appendix E. Reference policy proposal 47
Appendix F. Dedicated-browser root-store reconciliation 48
References 50

Page numbers refer to this publication rendering. In Word, navigation is also available through the document heading pane.

1. Problem statement and security objective

The original requirement can be stated informally as “Chrome must use only ca.scone.cloud for *.scone.cloud.” That wording contains three ambiguities: what “use” means when more than one valid path exists, whether the wildcard denotes one DNS label or an entire subtree, and how the CA is identified. This chapter removes those ambiguities and defines a testable acceptance rule.

1.1 Protected namespace

The protected namespace in this whitepaper is the DNS subtree rooted at scone.cloud. It includes the apex scone.cloud and names formed by adding labels to the left, such as api.scone.cloud and east.api.scone.cloud. The trust constraint is therefore written as scone.cloud. It is not written as *.scone.cloud: the latter is a wildcard certificate pattern, not the RFC 5280 syntax used for DNS name constraints. RFC 5280 states that a DNS name formed by adding zero or more labels to the left of the constraint satisfies the constraint. [1]

Do not constrain a CA to the public suffix “cloud” A permitted DNS name of cloud would authorize the added CA for every name below the public .cloud top-level domain. The safe scope is an organizational namespace such as scone.cloud or a narrower subtree such as secure.scone.cloud.

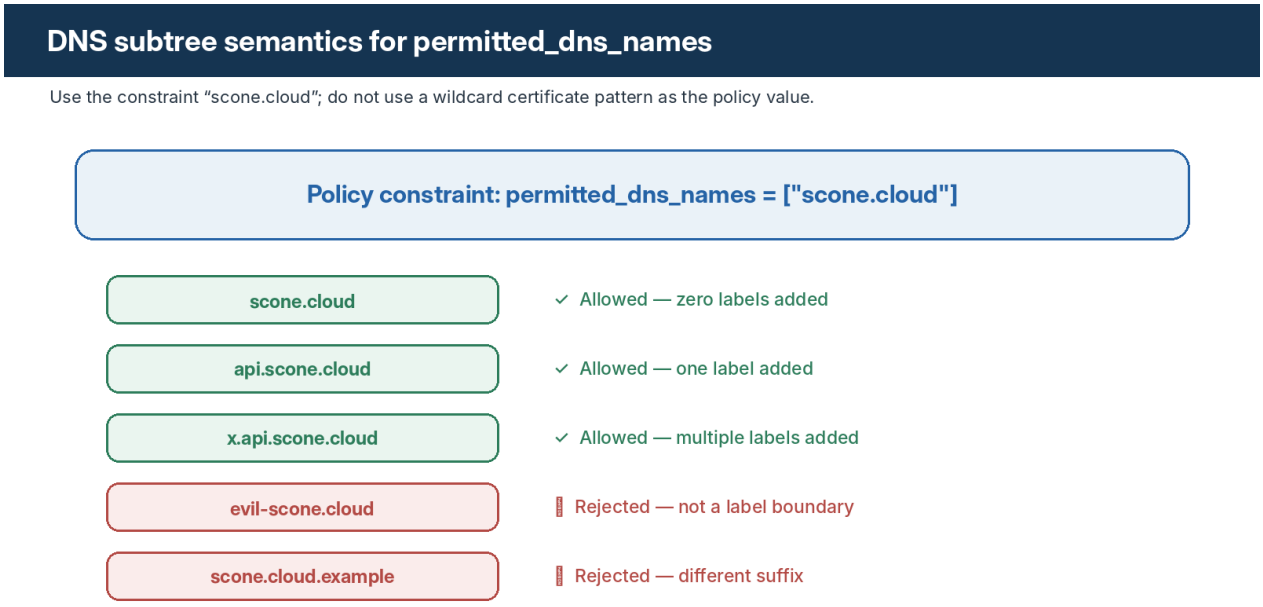


Figure 2 — Label-aware matching for the policy constraint scone.cloud.

1.2 CA identity

The string ca.scone.cloud may be a hostname at which CA services are published, a human-readable certificate subject, or both. None of those strings is a strong cryptographic identifier. The browser policy contains the actual DER-encoded CA certificate, and operational verification should record both the certificate SHA-256 fingerprint and the SHA-256 hash of its SPKI. The certificate fingerprint binds one exact certificate. The SPKI hash binds the public key across a controlled reissuance of the CA certificate and is therefore convenient for an exclusive-anchor policy and planned rotation.

1.3 Formal acceptance rule

Let h be the reference hostname, N the protected subtree scone.cloud, P a candidate certification path, Valid(P,h) the ordinary Chrome certificate-validity predicate, and Approved(P) a predicate that is true when P contains one of the organization-approved CA SPKIs. The required browser decision is:

PSEUDOCODE

```
Accept(h, certificate) = exists P: Valid(P, h)
                AND (h not in N OR Approved(P))
```

For a hostname outside N, ordinary Chrome PKI remains in effect. For a hostname inside N, ordinary validation is necessary but not sufficient: the verified path must also contain an approved CA key. The rule must be evaluated for every new TLS connection, including connections established for top-level navigation, subresources, WebSocket handshakes, service workers, preconnects and HTTP/3.

1.4 Threat model

Table 3 — Threats addressed by exclusive domain-to-CA binding

Threat	Security objective
Public-CA misissuance	A certificate issued by any public CA other than the approved CA must be rejected for a protected hostname.
Enterprise TLS interception	A locally or centrally installed interception CA must not authenticate the protected namespace unless explicitly approved.
DNS or routing redirection	Redirection to an attacker-controlled endpoint must not succeed without an approved certificate chain.
Alternate-chain construction	Cross-signing or AIA retrieval must not let the verifier select an unapproved trust anchor.
CA key rotation	A controlled overlap of old and new CA keys must be possible without a fail-open interval.
Policy tampering	A non-administrative user must not be able to remove the control or add a new trusted CA.

The managed-endpoint administrator remains in the trusted computing base unless the browser and policy are themselves measured and attested. An administrator who can replace the browser binary, modify machine policy or install a kernel-level interception component can generally bypass browser policy. The whitepaper therefore distinguishes browser-policy assurance from stronger measured-device assurance.

1.5 Scope and non-goals

- The report concerns TLS server authentication in Chrome and Chromium-derived enterprise deployments. Client-certificate selection and WebAuthn are discussed only where they complement server authentication.
- The report does not claim that the designated CA is trustworthy merely because it is private. CA key protection, issuance authorization and audit are separate requirements.
- The report does not treat Certificate Transparency, CAA or DNSSEC as equivalent to an exclusive browser policy. They provide different controls.
- The report does not recommend constraining a private CA to the entire .cloud public suffix.

2. Chrome certificate verification architecture

Chrome uses a certificate verifier that constructs and validates candidate certification paths. Chromium documents CertVerifyProcBuiltin as the cross-platform implementation that builds paths internally. Public trust anchors are supplied by the Chrome Root Store, while platform integration can contribute user and enterprise roots. Enterprise certificate policies add further anchors, constraints, intermediate hints and SPKI distrust entries. [2][3][4][11]

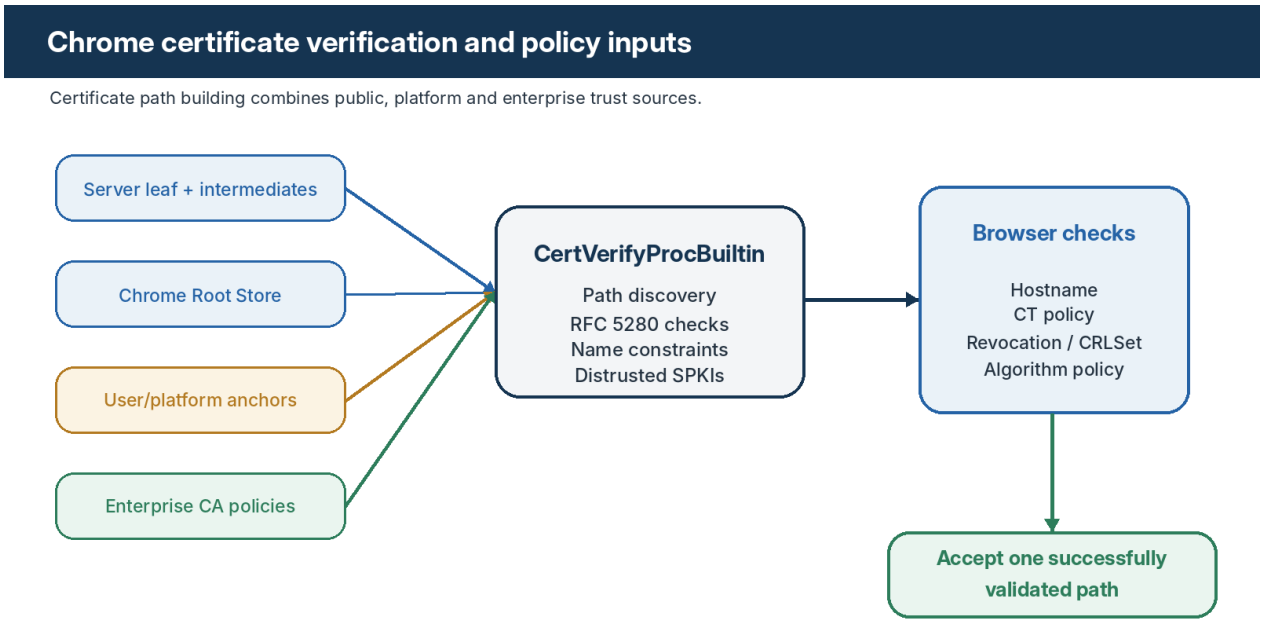


Figure 3 — Simplified Chrome certificate-verification pipeline and trust inputs.

2.1 Trust sources

Table 4 — Trust inputs relevant to the design

Source	Role	Relevant control
Chrome Root Store	Default public trust anchors distributed and updated by Chrome.	Not removable per hostname by current enterprise certificate policies.
User/platform trust store	User-added and enterprise-added local anchors imported from the operating system when enabled.	CAPlatformIntegrationEnabled can suppress this source on supported desktop platforms.
CACertificates	Enterprise-provided anchors. Current Chromium places them in the enforced-constraints anchor set.	Embedded anchor Name Constraints and NotBefore/NotAfter are enforced.
CACertificatesWithConstraints	Enterprise-provided anchors with policy-supplied permitted DNS names and/or CIDRs.	Use for a CA that does not carry the required constraint in its certificate.
CADistrustedCertificates	Global SPKI distrust entries.	Useful for emergency distrust; not scoped to a DNS namespace.
Intermediate / hint certificates	Certificates available to path construction but not necessarily trusted.	Do not confuse a hint with a trust anchor.

2.2 Path construction and the “any valid path” property

A server sends a leaf certificate and usually one or more intermediates. The verifier may also obtain intermediates from caches or Authority Information Access and may know cross-signed versions of the same CA. It then searches for a path that terminates at an eligible trust anchor. RFC 5280 defines validation of a candidate path; path discovery and anchor selection are local implementation matters. Chrome follows path-building guidance and can evaluate multiple possible paths. [1][11]

This architecture is deliberately flexible. It supports CA transitions, cross-signing and heterogeneous trust stores. The same flexibility creates the exclusivity problem: a constrained private path and an unconstrained public path can both be valid. Unless the effective trust-anchor set is filtered for the reference hostname, success of either path is sufficient.

2.3 Validation checks after or alongside path building

- Signature, key-usage, extended-key-usage, basic-constraints, algorithm and validity checks.
- Name Constraints processing for relevant name forms, including every DNS subjectAlternativeName in the subsequent certificates.
- Reference-hostname matching against the leaf certificate.
- Distrust and revocation processing, including CRLSet and optional online checks.
- Certificate Transparency policy for certificates and roots to which Chrome applies CT requirements.
- Browser-specific policy rules in addition to the result of an underlying platform verifier.

An exclusive-anchor check belongs at this level, not in page JavaScript or an extension. It must receive the canonical reference hostname and the final verified path, and it must affect the TLS handshake result before application data is delivered.

2.4 Chrome Root Store and platform integration

Modern Chrome uses the Chrome Root Store and Chrome Certificate Verifier on supported platforms. Chromium states that the built-in verifier relies on platform integration for user and enterprise configured roots while public roots come from the Chrome Root Store. The CAPlatformIntegrationEnabled enterprise policy controls whether user-added platform certificates are included when the Chrome Root Store is used. Disabling this policy reduces the number of local trust paths but does not remove public roots from the Chrome Root Store. [2][8][11]

> **Important consequence** CAPlatformIntegrationEnabled=false is valuable against locally installed interception roots, but it cannot make ca.scone.cloud exclusive because Chrome’s built-in public roots remain eligible.

3. Name Constraints and constrained certificate trust

Name Constraints answer the question "where may this CA be used?" They do not answer "which CAs may be used here?" That directionality is the central conceptual distinction in this whitepaper.

3.1 Embedded X.509 Name Constraints

RFC 5280 defines the critical Name Constraints extension for CA certificates. It contains permitted and/or excluded subtrees. Restrictions apply to subject names and subject alternative names in subsequent certificates. If a critical constraint applies to a name form that a client cannot process, the client must reject the path. [1]

A dedicated issuing CA for scone.cloud can therefore carry a critical permitted DNS subtree of scone.cloud. When a leaf certificate contains api.scone.cloud, that name is within the subtree. If the same leaf also contains unrelated.example in its DNS SAN list, the path through the constrained CA is invalid because the unrelated name is outside the permitted subtree. The rule is evaluated against all relevant DNS SANs, not only the hostname currently being visited.

OPENSSL X509 -TEXT EXCERPT

```
X509v3 Name Constraints: critical
  Permitted:
    DNS:scone.cloud
```

3.2 Chrome enforcement for enterprise anchors

Trust anchors are local configuration, and some PKI implementations historically treated constraints or validity on a self-signed root differently from constraints on an intermediate. Chromium explicitly distinguishes enterprise anchors whose embedded constraints and NotBefore/NotAfter fields are enforced. The CACertificates policy is currently mapped to this enforced-constraints anchor set. [3][4]

> **Design recommendation** Prefer a dedicated CA certificate for the protected namespace. If the CA certificate itself contains critical Name Constraints, distribute it through CACertificates so Chrome enforces the embedded constraint. If the CA certificate cannot be changed, use CACertificatesWithConstraints to attach the permitted subtree externally.

3.3 Externally supplied constraints

CACertificatesWithConstraints accepts a list of dictionaries containing a base64-encoded DER certificate and a constraints object. The current schema exposes permitted_dns_names and permitted_cidrs. At least one constraint is required. The policy is supported on Chrome desktop, ChromeOS and Android from version 132 and is dynamically refreshed per profile. [5]

JSON

```
{
 "CACertificatesWithConstraints": [
  {
   "certificate": "<BASE64_DER_CA>",
   "constraints": {
    "permitted_dns_names": ["scone.cloud"]
   }
  }
 ]
}
```

The policy currently does not expose an excluded_dns_names property. If exclusion logic is required, encode it in the CA certificate's critical X.509 Name Constraints extension or implement it in the exclusive enforcement component. External constraints are useful because the browser administrator can reduce the authority of an existing CA without reissuing the CA certificate.

3.4 Constraint semantics for the example namespace

Table 5 — Expected DNS Name Constraints behavior

Leaf DNS SAN	Valid through CA constrained to scone.cloud?	Reason

Leaf DNS SAN	Valid through CA constrained to scone.cloud?	Reason
scone.cloud	Yes	Zero labels are added to the left of the constraint.
api.scone.cloud	Yes	One label is added.
east.api.scone.cloud	Yes	Multiple labels are added.
evil-scone.cloud	No	The comparison is label-aware; the suffix is not a separate label.
scone.cloud.example	No	The DNS suffix is different.
api.scone.cloud + unrelated.example	No	Every DNS SAN must remain within the permitted subtree for this path.
IP address only	Not controlled by a DNS constraint	Add permitted_cidrs if the CA must also be constrained for IP identities.

3.5 Wildcard certificates are different

A leaf SAN of *.scone.cloud normally authenticates one label, such as api.scone.cloud, but not east.api.scone.cloud and not the apex scone.cloud. The CA constraint scone.cloud is broader: it describes the entire subtree. Do not infer the scope of a CA constraint from the scope of one wildcard leaf certificate. Issue explicit SANs or multiple wildcard levels only where operationally justified.

4. Chrome enterprise policies relevant to certificate trust

This chapter summarizes the policy set used by the runbooks. The policies are complementary. None provides a domain-scoped required-anchor allowlist.

Table 6 — Chrome policies used in the recommended deployment

Policy	Purpose	Security effect	Scope / caveat
CACertificatesWithConstraints	Add a trust anchor with external permitted DNS names or CIDRs.	Restricts the added CA to configured namespaces.	Per profile; additive trust.
CACertificates	Add enterprise trust anchors.	Current Chromium enforces embedded anchor constraints and validity.	Per profile; additive trust.
CADistrustedCertificates	Distrust certificates with listed SPKIs.	Blocks any path containing those keys.	Global for the profile; not hostname-scoped.
CAPlatformIntegrationEnabled	Include user-added platform anchors.	False reduces local and enterprise interception paths from the OS store.	Does not disable Chrome Root Store.
CACertificateManagementAllowed	Control user certificate management.	Value 2 prevents user management while still allowing viewing.	Administrative hardening.
RequireOnlineRevocationChecksForLocalAnchors	Hard-fail online OCSP/CRL checks under local anchors.	Improves revocation for the private CA when endpoints are reachable.	Can cause availability failures; browser-level rather than per profile.
URLBlocklist / URLAllowlist	Restrict navigation and requests.	Useful for a dedicated browser profile.	Not a certificate-verification control; allowlist has format and entry limits.
ProxySettings / ProxyPacMandatory	Force a proxy configuration and prevent direct fallback when PAC fails.	Part of a proxy enforcement architecture.	Must be backed by OS/network egress controls.
QuicAllowed	Allow or disable QUIC/HTTP/3.	False forces TCP transports, for which locally added private trust anchors are considered.	Browser-level; restart and test. It is transport hardening, not domain-scoped CA binding.
ChromeRootStoreEnabled (retired transition policy)	Historically selected the Chrome Root Store or legacy platform verifier during migration.	No current supported empty-root-store or replace-root-store mechanism.	Do not design a production control around this retired switch. [23]

4.1 Recommended baseline policy bundle

JSON

```
{
 "CACertificatesWithConstraints": [
  {
   "certificate": "<BASE64_DER_CA>",
   "constraints": {
    "permitted_dns_names": ["scone.cloud"]
   }
  }
 ],
 "CAPlatformIntegrationEnabled": false,
 "CACertificateManagementAllowed": 2,
 "RequireOnlineRevocationChecksForLocalAnchors": true,
 "QuicAllowed": false
}
```

QuicAllowed=false is recommended whenever the deployment relies on a locally or enterprise-added private CA. Chromium documents that local trust decisions can add and remove trust for TCP TLS, whereas QUIC considers local decisions only for removing trust. Disabling QUIC avoids relying on a locally added anchor in HTTP/3 and makes transport behavior explicit. Because QuicAllowed is a browser-level control, restart Chrome and verify that no QUIC connection is established. [11][26]

Set RequireOnlineRevocationChecksForLocalAnchors only after validating that every managed endpoint can reach the CA's OCSP responders or CRL distribution points. Chromium treats inability to obtain status as a hard failure under that mode. A short-lived-certificate design may instead rely on short validity plus emergency key distrust, but that is a risk decision rather than a direct substitute for revocation. [3][10]

4.2 Policy processing in Chromium

Chromium's ProfileNetworkContextService reads the enterprise lists and serializes them to the certificate verifier service. CACertificates entries are added to trust_anchors_with_enforced_constraints. CACertificatesWithConstraints entries are parsed into certificate-plus-constraint objects and added to trust_anchors_with_additional_constraints. CADistrustedCertificates entries are converted to SPKI bytes, and CAPlatformIntegrationEnabled controls inclusion of the system trust store. [4]

This implementation confirms both the value and the limitation of the policy. The private CA is represented as another trust anchor with restrictions. There is no corresponding data structure in the current policy path that maps a reference hostname to the only permitted trust anchors.

4.3 Policy precedence and administration

Chrome policy can arrive from operating-system policy, machine cloud policy, user policy and other supported sources. The effective value depends on Chrome's documented policy precedence and management mode. Administrators should verify the source, scope and status on chrome://policy rather than assuming that a local JSON or registry value won. [19][20][21]

- Use mandatory machine-level policy for the trust configuration.
- Prevent non-administrative modification of the policy file, registry key or configuration profile.
- Treat a policy conflict or parse error as a deployment failure.
- Record the effective policy export as evidence during rollout and incident response.

5. Case study: ca.scone.cloud for the scone.cloud namespace

This case study presents two supported ways to limit the authority of the private CA. Both provide constrained additional trust. Neither creates exclusivity.

5.1 Option A — policy-supplied constraint

Use this option when the existing CA certificate has no suitable Name Constraints extension or when the browser administrator wants an independent restriction. Convert the CA certificate to DER, base64-encode the DER bytes without PEM markers, and deploy it through CACertificatesWithConstraints.

SHELL

```
openssl x509 -in ca.scone.cloud.pem -outform DER -out ca.scone.cloud.der
base64 < ca.scone.cloud.der | tr -d '\n' > ca.scone.cloud.der.b64

openssl x509 -in ca.scone.cloud.pem -noout \
  -subject -issuer -serial -dates -fingerprint -sha256
```

The permitted_dns_names entry must be scone.cloud. A wildcard token is unnecessary and may be interpreted as a literal invalid or unintended constraint depending on policy validation. Use lowercase A-label form for internationalized names and validate the result in chrome://policy.

5.2 Option B — embedded critical Name Constraints

Use a dedicated issuing CA whose certificate contains a critical permitted DNS subtree. Distribute that CA certificate through CACertificates. Current Chromium maps CACertificates to the trust-anchor set that enforces embedded anchor constraints and anchor validity. This design makes the limitation visible in the PKI object itself and is portable to other conforming clients that enforce the extension. [1][3][4]

OPENSSL CONFIG

```
[ v3_scone_issuing_ca ]
basicConstraints       = critical, CA:true, pathlen:0
keyUsage               = critical, keyCertSign, cRLSign
subjectKeyIdentifier   = hash
authorityKeyIdentifier = keyid:always, issuer
nameConstraints        = critical, permitted;DNS:scone.cloud
crlDistributionPoints  = URI:https://pki.scone.cloud/crl/scone-issuing.crl
authorityInfoAccess    = OCSP;URI:https://pki.scone.cloud/ocsp
```

> **Prefer a dedicated subordinate key** Do not reuse a broad corporate root key as the online issuer for service certificates. Use an offline root and a dedicated pathLen=0 issuing CA for scone.cloud, or directly trust that dedicated issuing CA in the managed browser profile.

5.3 Expected outcomes

Table 7 — Difference between constrained additional trust and exclusive binding

Scenario	Constrained-trust baseline	Exclusive target
api.scone.cloud issued by ca.scone.cloud	Accept	Accept
api.scone.cloud issued by an ordinary Chrome-trusted public CA	Accept if the public path is otherwise valid	Reject
unrelated.example issued by ca.scone.cloud	Reject through the constrained CA	Reject through the constrained CA
api.scone.cloud issued by a platform-installed interception CA; platform integration enabled	Accept if path is valid	Reject unless explicitly approved

Scenario	Constrained-trust baseline	Exclusive target
api.scone.cloud issued by a platform-installed interception CA; platform integration disabled	Usually reject that local path, but a public path remains possible	Reject

5.4 Why the CA certificate may still carry constraints

Even when a custom Chromium verifier or proxy ultimately provides exclusivity, the CA should remain constrained. The exclusive control protects the namespace from other CAs; the Name Constraints extension protects other namespaces from the private CA. The two controls operate in opposite directions and are complementary.

6. Can stock Chrome enforce that only the explicit CA is used?

Answer Not with a domain-scoped enterprise policy in current stock Chrome. Chrome can constrain an added CA and can globally distrust selected SPKIs, but it cannot currently express “for scone.cloud, the verified path must contain one of these CA keys.”

6.1 Counterexample

Assume the administrator installs ca.scone.cloud using CACertificatesWithConstraints with permitted_dns_names=["scone.cloud"]. The server api.scone.cloud first presents a leaf issued by that CA. Chrome builds the private path and accepts it. The server is then reconfigured to present a valid certificate issued by a public CA whose root is in the Chrome Root Store. Chrome can build a different valid path and accepts that path as well. No constraint on ca.scone.cloud applies to the public CA, and no current policy reserves the hostname for the private CA.

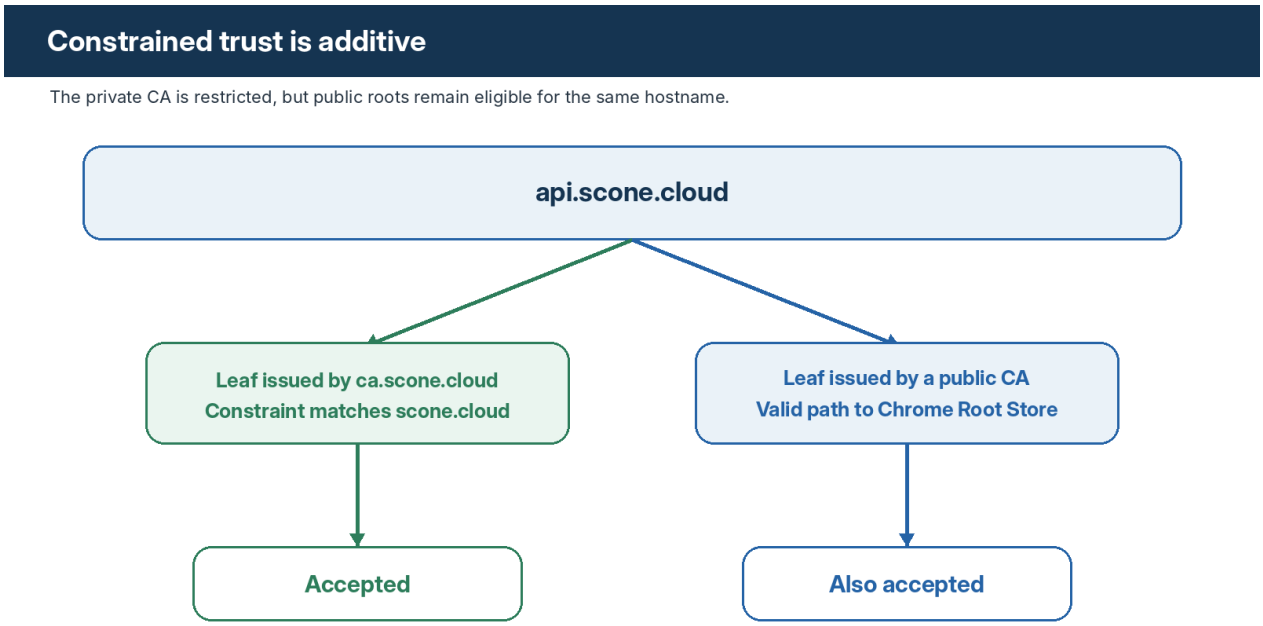


Figure 4 — Both a constrained private path and an ordinary public path can be accepted.

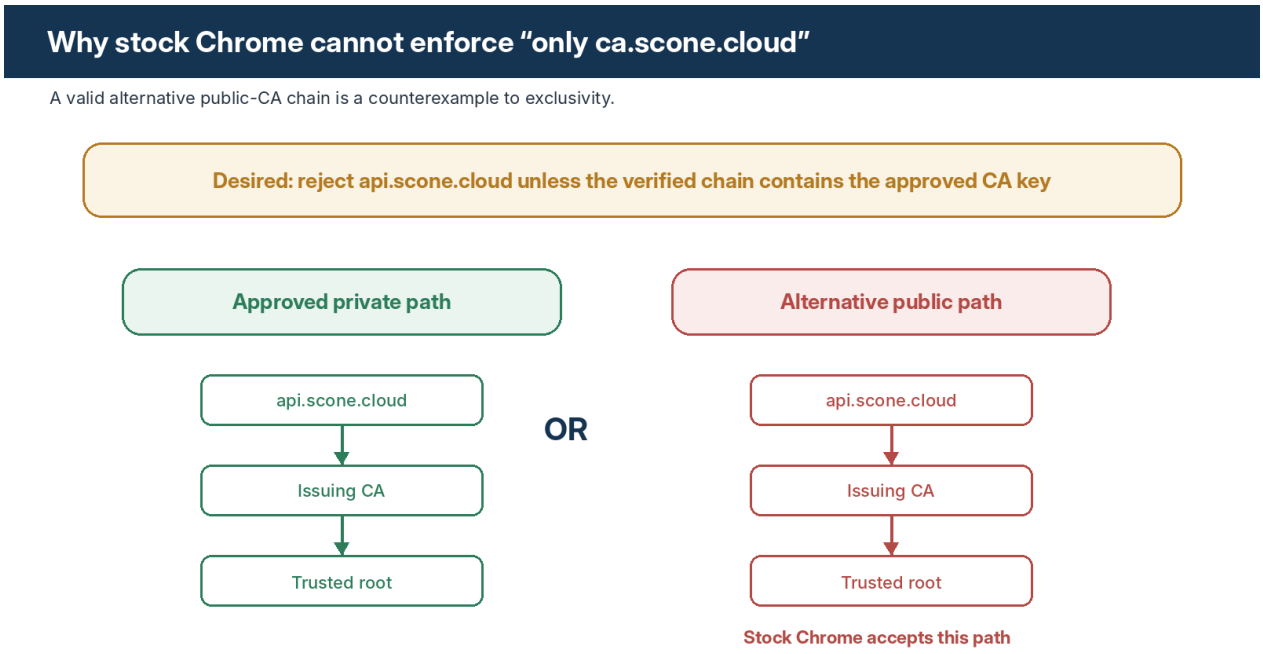


Figure 5 — A valid alternative public path disproves exclusive CA enforcement in stock Chrome.

6.2 Formal reasoning

Let T be the set of trust anchors eligible for a Chrome profile. Before policy deployment, T includes the Chrome Root Store and any enabled platform anchors. CACertificatesWithConstraints adds an anchor A plus a condition that limits paths terminating at A. The stock acceptance predicate remains:

PSEUDOCODE

```
Accept_stock(h, certificate) = exists P, t in T union {A}:
  ValidPath(P, certificate, h, t)
  AND (t != A OR ConstraintA(h, P))
```

The desired predicate additionally requires Approved(P) for h in N. Since stock Chrome has no term equivalent to that requirement, any path to an unapproved t in T is a counterexample. This is not an implementation bug in Name Constraints; it is a different policy primitive.

6.3 Why disabling platform integration is insufficient

CAPlatformIntegrationEnabled=false removes one class of alternate paths: user-added roots from the platform store. It does not remove roots in the Chrome Root Store and does not prevent a higher-precedence Chrome enterprise policy from adding another anchor. The setting should be part of the baseline because it reduces attack surface, but the security claim must remain precise. [8][11]

6.4 Could CADistrustedCertificates approximate an allowlist?

CADistrustedCertificates blocks any chain containing a listed SPKI. The proposal to “remove all roots” has two interpretations. Variant A leaves only ca.scone.cloud effectively trusted by globally distrusting every other active public and enterprise anchor. Variant B attempts to preserve ordinary browsing by re-adding public roots with an exclusion for scone.cloud. Variant A can approximate the security property for a single-purpose TCP-only profile; Variant B is not expressible with current stock-Chrome enterprise certificate policy. [5][7][11]

The distinction matters. A complete effective-anchor allowlist can be secure, but stock Chrome implements it only indirectly through a global SPKI denylist. The guarantee therefore depends on a complete and current inventory, correct policy precedence, disabled platform integration, absence of other enterprise-added anchors, and use of TCP-based TLS. It is not equivalent to a native rule that selects trust anchors from the reference hostname.

> Practical conclusion A complete global-distrust profile can enforce the approved CA for TCP-based TLS only when it is intentionally single-purpose, QUIC is disabled, and the active root-store snapshot is continuously reconciled. It cannot preserve general public browsing. For a general managed browser, implement host-scoped anchor filtering or use a non-bypassable proxy.

6.5 Variant A — a dedicated browser with only ca.scone.cloud effectively trusted

For a dedicated application browser, the effective trust-anchor set can be reduced to the approved CA by globally distrusting every other root key known to the profile. The approved CA is then added through CACertificatesWithConstraints with permitted_dns_names=["scone.cloud"] or through CACertificates when the certificate itself carries a critical Name Constraints extension. For TCP TLS, a complete configuration has the following outcomes:

- api.scone.cloud chaining to ca.scone.cloud is accepted when all ordinary validation checks pass.
- api.scone.cloud chaining only to a public root is rejected because that root SPKI is distrusted.
- unrelated.example chaining to ca.scone.cloud is rejected by the permitted-subtree constraint.
- A platform-installed interception CA is ineligible when platform integration is disabled and its key is not added elsewhere.
- Ordinary public HTTPS fails by design because its roots are also globally distrusted.

CONCEPTUAL POLICY — THE FULL ROOT LIST IS GENERATED, NOT WRITTEN BY HAND

```
{
 "CADistrustedCertificates": [
  "<BASE64_DER_CHROME_ROOT_1>",
  "<BASE64_DER_CHROME_ROOT_2>",
  "<...every other active public and enterprise anchor...>"
 ],
 "CACertificatesWithConstraints": [
  {
   "certificate": "<BASE64_DER_CA_SCONE_CLOUD>",
   "constraints": {
    "permitted_dns_names": ["scone.cloud"]
   }
  }
 ],
 "CAPlatformIntegrationEnabled": false,
 "CACertificateManagementAllowed": 2,
 "QuicAllowed": false,
 "URLBlocklist": ["*"],
 "URLAllowlist": [
  "https://scone.cloud",
  "https://[*.]scone.cloud"
 ]
}
```

Security condition. The claim “only ca.scone.cloud is accepted” is valid only while the distrust set covers every other active trust-anchor SPKI and no higher-precedence policy adds another anchor. Treat an inventory mismatch as a security failure, not as a maintenance warning.

6.6 Current Chrome has no supported empty-root-store switch

The Chrome Root Store is the browser’s built-in source of public trust. The transitional ChromeRootStoreEnabled policy was provided during migration from platform verifiers and is not a current supported mechanism for replacing the Chrome Root Store with an empty organization-owned store. Stock Chrome therefore cannot simply be configured with “public roots = none.” The enterprise-policy approximation is to leave the store present and globally distrust every active root SPKI through CADistrustedCertificates. [11][23]

This difference is operationally important. A custom Chromium build can compile a deliberately different trust store, but once an organization maintains a custom build, hostname-scoped anchor filtering is safer and more usable than compiling a browser that trusts only one CA globally. For stock Chrome, do not rely on obsolete command-line flags or retired policies to suppress the root store.

Policy delivery and browser dependencies also need review. Chrome cloud management, extension updates, sign-in, Safe Browsing-related services, identity-provider redirects and support pages may depend on ordinary public HTTPS. A dedicated profile must either avoid those dependencies, make them reachable through an explicitly designed trust path, or accept that they fail. Deliver the critical trust policy through a channel that does not itself depend solely on public HTTPS inside the restricted profile.

6.7 Root-store updates make the denylist a moving security boundary

Chrome updates the Chrome Root Store as a component. Chromium documents routine updates of approximately quarterly and notes that an existing Chrome installation may receive a newly released root-store component within about 24 hours. A root added after the enterprise distrust snapshot becomes eligible immediately unless its SPKI is also present in policy. [11]

REQUIRED COVERAGE INVARIANT

```
ActivePublicAnchorSPKIs - DistrustedSPKIs = empty set
ActiveEnterpriseAnchorSPKIs - {ApprovedCA} - DistrustedSPKIs = empty set
PlatformIntegrationEnabled = false
QUICEnabled = false
EffectiveApprovedAnchors = {ca.scone.cloud}
```

The deployment pipeline should record the Chrome version, the active chrome_root_store component version, the certificate SHA-256 fingerprints and the derived SPKI SHA-256 hashes. Chrome exposes the active component through chrome://system; Chromium also publishes a current root-store manifest. The manifest's certificate fingerprints are inventory identifiers, while CADistrustedCertificates makes its decision by SPKI, so the pipeline must obtain the DER certificate and calculate the SPKI hash rather than comparing only certificate fingerprints. [7][11][27]

- Generate the distrust policy from a pinned, reviewed root certificate corpus; do not hand-maintain it.
- Gate every Chrome binary and root-store component update on a coverage comparison and the negative-certificate test.
- Block protected-service access when the active root-store version is unknown or newer than the approved policy snapshot.
- Test the policy-size and delivery limits of GPO, MDM and Chrome cloud management before relying on a full-root list.
- Do not freeze root-store updates indefinitely; doing so also delays security removals and other trust corrections.

6.8 Variant B — re-add public roots while excluding scone.cloud

The ideal general-browser model would trust each ordinary public root for every namespace except scone.cloud and trust ca.scone.cloud only for scone.cloud. X.509 Name Constraints can represent excludedSubtrees, so the model is conceptually valid. It would preserve normal public HTTPS while reserving the protected subtree for the private CA. [1]

DESIRED TRUST MODEL

```
For h within scone.cloud:
  eligible anchors = {ca.scone.cloud}

For h outside scone.cloud:
  eligible anchors = ordinary Chrome roots

For ca.scone.cloud:
  permitted DNS subtree = scone.cloud
```

Current CACertificatesWithConstraints does not expose an excluded_dns_names property. Its schema supports only permitted_dns_names and permitted_cidrs. Chromium's current Chrome Root Store metadata similarly defines a positive permitted_dns_names field, not an enterprise-controlled arbitrary DNS exclusion list. Therefore an administrator cannot attach "everything except scone.cloud" to each public root through stock Chrome policy. [5][25]

Enumerating the complement as positive permitted subtrees is not stable: preserving the public Web would require continuously listing all acceptable namespaces, including changing siblings under .cloud. Reissuing a valid public-root certificate with a new embedded exclusion would require control of the CA key. An administrator-modified certificate with an invalid self-signature is unsupported and should not be used as a production trust object.

6.9 Distrust followed by re-trust of the same SPKI does not work

A tempting sequence is to globally distrust a public root, then add a second copy of that root with constraints. CADistrustedCertificates intentionally matches SubjectPublicKeyInfo rather than the complete certificate encoding. Any certificate containing the same public key is distrusted. Chromium's enterprise certificate browser test TestDistrustOverridesTrust places the same certificate in both trust and distrust policy and expects validation to fail. [7][24]

UNSUPPORTED COMBINATION

```
CADistrustedCertificates: [ public_root_R ]
CACertificatesWithConstraints: [ public_root_R + constraints ]

Result: public_root_R remains distrusted because the SPKI matches.
```

The result is unchanged when the certificate is re-encoded or reissued with the same key: the SPKI match still triggers distrust. Thus global distrust cannot be used as a "remove, then re-add with narrower trust" primitive.

6.10 QUIC and locally added private trust

Chromium documents a transport asymmetry: for TCP TLS, local trust decisions may add or remove trust; for QUIC, local decisions are considered only for removing trust. A ca.scone.cloud anchor added by enterprise policy is therefore not a safe basis for an HTTP/3 connection. Chrome may fail the QUIC attempt or fall back to TCP, but the deployment must not depend on implicit fallback. [11]

REQUIRED POLICY FOR THE STOCK PRIVATE-CA DESIGN

```
{
  "QuicAllowed": false
}
```

Restart the browser after deploying the policy and verify through NetLog and network telemetry that no UDP/443 QUIC connection is used. A custom Chromium build may deliberately add the approved CA to the verifier used for QUIC, and a proxy architecture may terminate or block QUIC, but both cases require explicit conformance testing. [26]

6.11 Why an extension is not a verifier replacement

The webRequest API can observe and, for policy-installed extensions, block many HTTP and WebSocket handshake requests. The API is an abstraction above the network stack, hides some browser-sensitive requests, and does not expose a general hook that replaces certificate path validation for every connection. It also does not intercept individual messages on an established WebSocket. A browser extension can add telemetry or a secondary block decision, but it should not be the security boundary for exclusive CA enforcement. [16]

7. Evaluation of alternative mechanisms

Table 8 — Mechanism comparison

Mechanism	Prevents an unapproved valid CA path?	Primary value	Assessment
Chrome constrained CA policy	No	Limits where the private CA can be used.	Required baseline.
Embedded Name Constraints	No	Limits the CA cryptographically in conforming clients.	Strongly recommended for the private issuing CA.
CAA DNS records	No at browser validation time	Tells compliant public CAs which issuers are authorized before issuance.	Enable as issuance defense in depth.
Certificate Transparency monitoring	No	Detects publicly logged unexpected issuance and provides audit evidence.	Enable monitoring; do not treat as prevention.
DANE/TLSA	Potentially, in a client that validates DNSSEC and TLSA	Relying-party binding of service to keys or CAs.	Not a stock Chrome enterprise enforcement mechanism.
HPKP	Historically yes for pins learned or preloaded	Public-key pinning.	Dynamic HPKP was removed; do not reintroduce its failure modes through headers.
Browser extension	Not reliably	Request observation and secondary blocking.	Insufficient as sole verifier.
Mandatory TLS policy proxy	Yes, if every path is forced through it	Central upstream certificate-policy enforcement.	Recommended enterprise option.
Custom Chromium verifier	Yes	Browser-local hostname-scoped anchor filtering.	Highest-assurance browser solution.
Complete global distrust in a dedicated profile	Conditionally, for TCP TLS while the list covers every other anchor	Reduces the effective trust set to the approved private CA.	Single-purpose fallback only; breaks public HTTPS, disable QUIC, continuously reconcile root updates.

7.1 CAA

CAA records allow a domain holder to identify CAs authorized to issue certificates. RFC 8659 explicitly distinguishes CAA, which is checked by CAs before issuance, from TLSA/DANE, which is a relying-party verification mechanism. The RFC also states that relying parties must not use current CAA records as part of certificate validation. CAA therefore reduces the probability of misissuance but does not make Chrome reject an already issued valid certificate. [13]

DNS ZONE EXAMPLE

```
scone.cloud.  CAA 0 issue "ca.scone.cloud"
scone.cloud.  CAA 0 issuewild "ca.scone.cloud"
scone.cloud.  CAA 0 iodef "mailto:pki-security@scone.cloud"
```

For a private CA, the issuer-domain value is meaningful only when the CA defines and documents it. If public issuance must be prohibited entirely, use the empty issue value or appropriate CAA policy, and verify behavior with each public CA ecosystem relevant to the domain.

7.2 Certificate Transparency

Certificate Transparency provides public append-only logs that allow domain owners and the ecosystem to detect suspect issuance. RFC 9162 and Chromium documentation emphasize that CT makes issuance auditable; the logs do not themselves prevent misissuance. Chrome applies CT policy to relevant publicly trusted certificates, but a private

CA certificate may not be accepted by public CT logs. For private PKI, deploy an internal transparency or issuance audit log if equivalent evidence is required. [14][15]

7.3 DANE/TLSA

DANE/TLSA can express relying-party constraints based on DNSSEC-protected records, including associations with a specific certificate or public key. It is conceptually close to the desired property. However, stock Chrome does not expose an enterprise policy that enables DANE validation for ordinary HTTPS connections. A local proxy or custom Chromium build could implement TLSA validation, but that becomes another form of custom enforcement rather than a stock configuration.

7.4 HPKP

HTTP Public Key Pinning allowed a site to advertise accepted public keys. Chromium removed dynamic HPKP after low adoption and operational incidents in which bad or hostile pins could cause long-lived denial of service. A modern enterprise design should use centrally managed, revocable policy with overlap keys and break-glass governance rather than a site-controlled long-lived header. [17]

7.5 Mandatory TLS policy proxy

A proxy can enforce the exact upstream rule by performing ordinary certificate validation and then checking whether the validated chain contains an approved SPKI when the CONNECT target or upstream SNI/reference host is within scone.cloud. This succeeds only if the endpoint cannot connect directly to destination IP addresses and cannot bypass the proxy over QUIC or another network path. The proxy-to-service path is the enforcement boundary; the browser-to-proxy path must also be authenticated and protected.

7.6 Custom Chromium verifier

A custom Chromium build can filter the trust-anchor set before path construction. For a protected hostname, only anchors whose SPKIs are on the required list are eligible. Ordinary RFC 5280, hostname, revocation, algorithm and CT processing still applies. This is the cleanest browser-local implementation because it reuses the browser's network stack for top-level navigation, HTTP/2, HTTP/3, WebSockets and service-worker traffic.

8. Recommended architecture

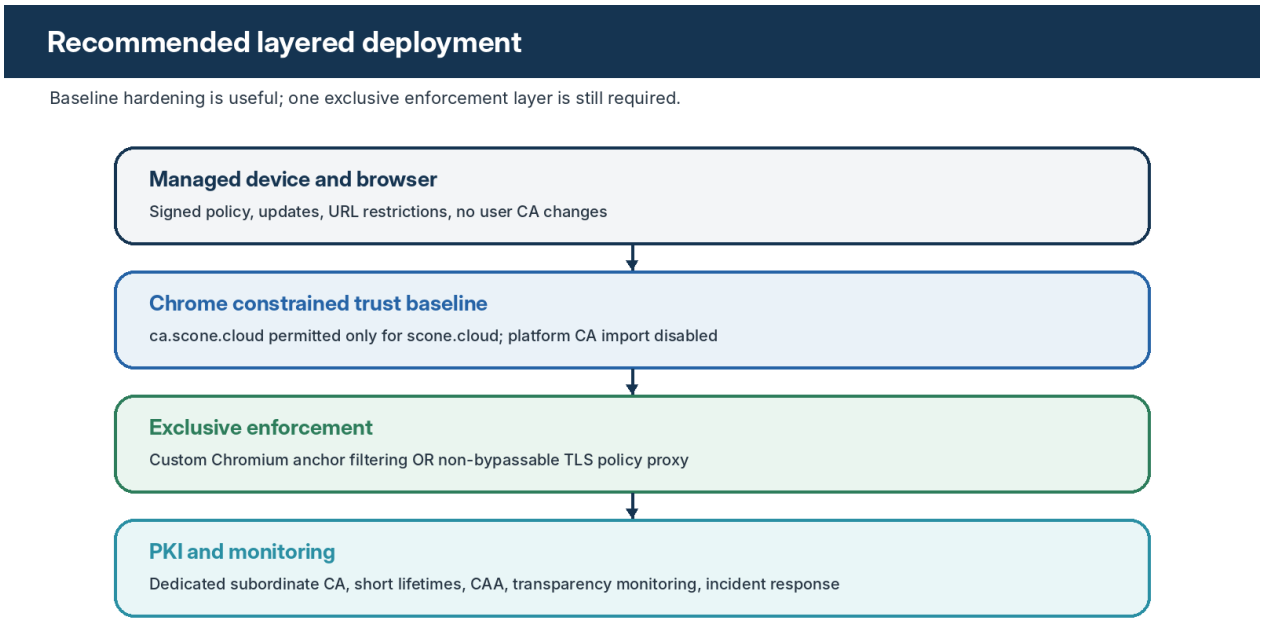


Figure 6 — Layered deployment combining baseline hardening and one exclusive enforcement point.

8.1 Baseline for every managed deployment

- Use a dedicated organizational namespace such as scone.cloud, not the public suffix cloud.
- Use a dedicated issuing CA key and identify it by controlled certificate and SPKI fingerprints.
- Constrain the CA with a critical embedded Name Constraints extension and/or CACertificatesWithConstraints.
- Set CAPlatformIntegrationEnabled=false unless a documented enterprise requirement depends on platform roots.
- Set CACertificateManagementAllowed=2 to prevent user changes.
- Evaluate RequireOnlineRevocationChecksForLocalAnchors with a staged availability test.
- Publish CAA and monitor public CT logs; maintain an internal issuance log for the private CA.
- Restrict the dedicated browser profile to required application and identity-provider URLs where feasible.
- Disable QUIC when Chrome relies on a locally or enterprise-added private CA, unless a tested custom verifier provides the same trust decision for HTTP/3.
- Continuously run both positive and negative certificate tests.

8.2 Select an exclusive enforcement point

Table 9 — Architecture selection

Environment	Preferred enforcement	Rationale
Highest-assurance dedicated browser	Custom Chromium trust-anchor filtering	The decision is made in the TLS verifier and cannot be bypassed by application-layer behavior.
Managed enterprise desktops with existing secure web gateway	Mandatory TLS policy proxy plus egress controls	Central deployment and logging; no browser fork, but proxy and network become trusted.
Single-purpose kiosk with no general web access	Custom Chromium; complete global-distrust profile only as controlled fallback	Global distrust can work for TCP when every other active SPKI is covered and QUIC is disabled, but the maintenance invariant is fragile. A verifier change remains cleaner.

Environment	Preferred enforcement	Rationale
Unmanaged consumer endpoints	No reliable stock-Chrome guarantee	Use ordinary public PKI, CAA/CT and application-level risk controls; exclusive browser policy cannot be assumed.

8.3 Dedicated-browser hardening

When the dedicated profile uses the complete-global-distrust fallback, treat root-store reconciliation as part of the browser update transaction. Do not promote a new Chrome or root-store component until the generated distrust policy and the alternate-public-CA negative test both pass.

A separate managed profile or branded browser reduces accidental policy interaction. Set URLBlocklist to * and allow only the protected application, identity provider, update service, OCSP/CRL endpoints and other documented dependencies. Disable developer tools and extension installation where appropriate. URL filtering is not a substitute for certificate enforcement, but it reduces the blast radius if the profile uses a private CA. [18]

> **Dependency warning** A strict URL allowlist can break authentication redirects, OCSP, CRL, font, telemetry or update endpoints. Capture the dependency graph during a pilot and keep the allowlist explicit rather than weakening the trust policy.

9. Operational runbook

This chapter is written as an implementation and operations checklist. Runbook steps marked “baseline” apply to stock managed Chrome. Steps marked “exclusive” apply only when the proxy or custom-verifier architecture is deployed.

9.1 Pre-deployment decisions

Table 10 — Pre-deployment decision record

Decision	Required record
Protected namespace	scone.cloud or narrower subtree; include exact apex/subdomain semantics.
Approved CA identity	DER certificate SHA-256 fingerprint and SPKI SHA-256 hash.
Constraint location	Embedded critical Name Constraints, external Chrome constraint, or both.
Revocation model	OCSP/CRL hard-fail, short-lived certificates, emergency SPKI distrust, or a documented combination.
Exclusive enforcement	Custom Chromium or mandatory proxy; name the owner and fail-closed behavior.
Break-glass process	Signed, time-limited policy with two-person approval and full logging.
Rotation plan	Old/new SPKI overlap period and test criteria.
Evidence retention	Policy exports, certificate chains, NetLogs, proxy logs and issuance audit records.
Transport policy	For a locally added private CA, set QuicAllowed=false or document a custom verifier that covers QUIC.
Global-distrust fallback	If used, record the active Chrome/root-store version, generated SPKI coverage, policy-delivery limit and fail-closed update gate.

9.2 Prepare and verify the CA certificate

SHELL

```
set -euo pipefail
CA_PEM=ca.scone.cloud.pem
CA_DER=ca.scone.cloud.der

openssl x509 -in "$CA_PEM" -noout \
  -subject -issuer -serial -dates -fingerprint -sha256

openssl x509 -in "$CA_PEM" -outform DER -out "$CA_DER"
base64 < "$CA_DER" | tr -d '\n' > "${CA_DER}.b64"

# SPKI pin in the common sha256/<base64> notation
SPKI=$(openssl x509 -in "$CA_PEM" -pubkey -noout | \
  openssl pkey -pubin -outform DER | \
  openssl dgst -sha256 -binary | openssl base64 -A)
printf 'sha256/%s\n' "$SPKI"

# Inspect embedded constraints
openssl x509 -in "$CA_PEM" -noout -text | \
  sed -n '/X509v3 Name Constraints/,+8p'
```

- Compare the fingerprint with an out-of-band value approved by the PKI owner.
- Reject a certificate whose basicConstraints does not assert CA:TRUE or whose keyUsage lacks keyCertSign.

- Record the exact certificate and SPKI in version control or a signed policy bundle.
- Do not identify the CA only by Subject or Common Name.
- If the CA is a directly trusted subordinate, confirm pathLen, issuance scope and revocation behavior.

9.3 Issue a dedicated name-constrained CA

The following OpenSSL configuration fragment creates the extension profile for a dedicated pathLen=0 issuing CA. The offline root signs the CSR using this profile. Protect the root offline and place the issuing key in an HSM or equivalent controlled service.

OPENSSL.CNF

```
[ ca ]
default_ca = root_ca

[ root_ca ]
dir             = ./pki
certificate     = $dir/root-ca.pem
private_key     = $dir/private/root-ca.key
new_certs_dir   = $dir/newcerts
database        = $dir/index.txt
serial          = $dir/serial
default_md      = sha256
policy          = policy_strict
copy_extensions  = none

[ policy_strict ]
commonName           = supplied
organizationName     = supplied
countryName          = optional

[ v3_scone_issuing_ca ]
basicConstraints       = critical, CA:true, pathlen:0
keyUsage               = critical, keyCertSign, cRLSign
subjectKeyIdentifier   = hash
authorityKeyIdentifier  = keyid:always, issuer
nameConstraints        = critical, permitted;DNS:scone.cloud
crlDistributionPoints   = URI:https://pki.scone.cloud/crl/scone-issuing.crl
authorityInfoAccess    = OCSP;URI:https://pki.scone.cloud/ocsp
```

SHELL

```
openssl ca -config openssl.cnf \
  -extensions v3_scone_issuing_ca \
  -in scone-issuing-ca.csr \
  -out scone-issuing-ca.pem \
  -days 1825 -notext -batch

openssl verify -CAfile root-ca.pem scone-issuing-ca.pem
openssl x509 -in scone-issuing-ca.pem -noout -text
```

> **Interoperability test** Test the exact CA chain with the Chrome versions and platforms in scope. Name Constraints interoperability is much better than historically, but a critical extension is intentionally fail-closed in clients that cannot process it.

9.4 Linux deployment — stock Chrome baseline

Google Chrome reads mandatory policy files from /etc/opt/chrome/policies/managed. Chromium distributions commonly use /etc/chromium/policies/managed. The directory and file must be writable only by administrators. [20]

SHELL

```
sudo install -d -o root -g root -m 0755 \
  /etc/opt/chrome/policies/managed

BASE64_DER_CA=$(tr -d '\r\n ' < ca.scone.cloud.der.b64)

sudo tee /etc/opt/chrome/policies/managed/scone-certificate-trust.json \
  >/dev/null <<JSON
{
  "CACertificatesWithConstraints": [
    {
      "certificate": "${BASE64_DER_CA}",
      "constraints": {
        "permitted_dns_names": ["scone.cloud"]
      }
    }
  ],
  "CAPlatformIntegrationEnabled": false,
  "CACertificateManagementAllowed": 2,
  "RequireOnlineRevocationChecksForLocalAnchors": true,
  "QuicAllowed": false
}
JSON

sudo chown root:root \
  /etc/opt/chrome/policies/managed/scone-certificate-trust.json
sudo chmod 0644 \
  /etc/opt/chrome/policies/managed/scone-certificate-trust.json
python3 -m json.tool \
  /etc/opt/chrome/policies/managed/scone-certificate-trust.json >/dev/null
```

Open chrome://policy, select Reload policies and verify that every policy has status OK. Because the certificate policy is per profile, verify the intended managed profile rather than an unmanaged personal profile. Restart Chrome: QuicAllowed is a browser-level transport setting, and existing network contexts and connections must not confuse the test result.

9.5 Windows deployment — Group Policy or MDM

Use Google's current Chrome ADMX templates or the organization's MDM. The complex certificate policy is represented as a JSON string. The direct registry example below is suitable for a controlled lab and for understanding the effective data; production deployment should use the organization's supported management channel. [19]

POWERSHELL

```
$ErrorActionPreference = 'Stop'
$Base = 'HKLM:\SOFTWARE\Policies\Google\Chrome'
New-Item -Path $Base -Force | Out-Null

$CaB64 = (Get-Content .\ca.scone.cloud.der.b64 -Raw) -replace '\s',''
$Constrained = @(
  @{
    certificate = $CaB64
    constraints = @{ permitted_dns_names = @('scone.cloud') }
  }
) | ConvertTo-Json -Depth 5 -Compress

New-ItemProperty -Path $Base -Name 'CACertificatesWithConstraints' `
  -PropertyType String -Value $Constrained -Force | Out-Null
New-ItemProperty -Path $Base -Name 'CAPlatformIntegrationEnabled' `
  -PropertyType DWord -Value 0 -Force | Out-Null
New-ItemProperty -Path $Base -Name 'CACertificateManagementAllowed' `
  -PropertyType DWord -Value 2 -Force | Out-Null
New-ItemProperty -Path $Base -Name 'RequireOnlineRevocationChecksForLocalAnchors' `
  -PropertyType DWord -Value 1 -Force | Out-Null
New-ItemProperty -Path $Base -Name 'QuicAllowed' `
  -PropertyType DWord -Value 0 -Force | Out-Null

Get-ItemProperty -Path $Base | Format-List
```

- Confirm whether policy is machine or user scoped and whether a cloud policy source overrides the registry value.
- Use chrome://policy to inspect the actual JSON value, scope and source.
- Protect the registry key with normal administrative ACLs and monitor it for changes.
- Test the deployment on a machine that also has the enterprise TLS-inspection root installed; platform integration should be disabled for the managed profile.

9.6 macOS deployment — configuration profile

Deploy the preferences in the managed com.google.Chrome domain using MDM. Do not rely on an unmanaged user defaults command for production. The mobileconfig excerpt below shows the relevant payload dictionary; replace the UUIDs and base64 value. [21]

MOBILECONFIG

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
 "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
 <key>PayloadType</key><string>Configuration</string>
 <key>PayloadVersion</key><integer>1</integer>
 <key>PayloadIdentifier</key><string>cloud.scone.chrome.trust</string>
 <key>PayloadUUID</key><string>REPLACE-WITH-UUID</string>
 <key>PayloadDisplayName</key><string>SCONE Chrome Trust Policy</string>
 <key>PayloadContent</key>
 <array>
  <dict>
   <key>PayloadType</key><string>com.google.Chrome</string>
   <key>PayloadVersion</key><integer>1</integer>
   <key>PayloadIdentifier</key><string>cloud.scone.chrome.trust.payload</string>
   <key>PayloadUUID</key><string>REPLACE-WITH-SECOND-UUID</string>
   <key>PayloadDisplayName</key><string>Chrome Certificate Trust</string>
   <key>CACertificatesWithConstraints</key>
   <array>
    <dict>
     <key>certificate</key><string>BASE64_DER_CA</string>
     <key>constraints</key>
     <dict>
      <key>permitted_dns_names</key>
      <array><string>scone.cloud</string></array>
     </dict>
    </dict>
   </array>
   <key>CAPlatformIntegrationEnabled</key><false/>
   <key>CACertificateManagementAllowed</key><integer>2</integer>
   <key>RequireOnlineRevocationChecksForLocalAnchors</key><true/>
   <key>QuicAllowed</key><false/>
  </dict>
 </array>
</dict>
</plist>
```

After MDM installation, inspect the profile in System Settings, then verify the effective values in chrome://policy. Test both a managed and an unmanaged profile to confirm that policy scoping matches the intended design.

9.7 Chrome Enterprise cloud deployment

- Create a pilot organizational unit or browser group.
- Search for each policy by its exact name and set it as mandatory.
- Upload or paste the CA certificate in the format required by the Admin console and configure permitted DNS names as scone.cloud.
- Disable platform integration and user CA management for the pilot group. Disable QUIC when the service chain depends on a locally added private CA.
- Export the effective policy or capture chrome://policy evidence from a test endpoint.
- Promote in stages only after the negative test using an alternate public CA has produced the expected baseline result and the exclusive layer has blocked it.

9.8 Verification procedure

Table 11 — Verification checklist

Step	Procedure	Pass criterion
Policy load	Open chrome://policy, click Reload policies, inspect each relevant policy.	Status OK; mandatory scope and expected source.
CA identity	Compare the deployed certificate fingerprint and SPKI with the approved values.	Exact match.

Step	Procedure	Pass criterion
Root store	Open chrome://system and inspect the chrome_root_store component where available.	Understand which public roots remain eligible.
Positive TLS test	Visit a host issued by ca.scone.cloud within scone.cloud.	Connection succeeds and chain displays the approved CA.
Constraint negative test	Present a certificate from ca.scone.cloud for unrelated.example.	Connection fails.
Alternate-CA baseline test	Present a valid public-CA certificate for api.scone.cloud.	Stock baseline usually succeeds; this demonstrates the limitation.
Exclusive test	Repeat alternate-CA test through custom Chromium or proxy.	Connection fails with a distinct policy error.
Transport variants	Test HTTP/2, WebSocket reconnect and service-worker fetch. For a local private CA, set QuicAllowed=false and verify that no HTTP/3 connection is used.	All accepted connections use TCP and the intended CA decision; no QUIC bypass.
Diagnostics	Capture chrome://net-export and server chain.	Evidence identifies selected path and policy decision.
Root-store coverage (fallback only)	Compare the active chrome_root_store version and all active anchor SPKIs with the generated CADistrustedCertificates list.	Every non-approved active anchor is distrusted; any mismatch blocks rollout.

> **Do not use curl as the browser-policy oracle** curl and OpenSSL are useful for server-chain diagnostics, but they generally use a different trust store and do not evaluate Chrome enterprise certificate policy. The authoritative test is Chrome itself, supported by NetLog and policy evidence.

9.9 Build a negative-test lab

Create two independent issuing CAs: approved-ca and alternate-ca. Issue two leaf certificates with the same SAN test.scone.cloud. Configure two test endpoints or two ports. Install only approved-ca through the Chrome policy. For the stock baseline, separately make alternate-ca eligible through the Chrome Root Store equivalent in the lab or a controlled enterprise anchor; for the exclusive layer, verify that the same alternate chain is rejected.

SHELL

```
# Inspect what each endpoint serves
openssl s_client -connect test.scone.cloud:443 \
  -servername test.scone.cloud -showcerts </dev/null

# Capture the leaf and compute its SPKI
openssl s_client -connect test.scone.cloud:443 \
  -servername test.scone.cloud </dev/null 2>/dev/null | \
  openssl x509 -pubkey -noout | \
  openssl pkey -pubin -outform DER | \
  openssl dgst -sha256 -binary | openssl base64 -A
```

- Use unique hostnames for the lab to avoid HSTS, cached sockets and service-worker state from production.
- Close all tabs and restart the browser between path variants during initial diagnosis.
- Clear or isolate profile caches when testing AIA and cross-signed paths.
- Record the exact certificate chain and browser version with every result.

9.10 Runbook for a dedicated global-distrust browser

Classification: specialized fallback. Use this runbook only for a dedicated browser/profile that does not require ordinary public Web access. Prefer a custom Chromium required-anchor rule for a long-lived production design.

- Isolate the browser profile or application and document every required application, identity, update, revocation and support endpoint.
- Record the Chrome binary version and the active chrome_root_store component version from chrome://system.
- Obtain the exact DER certificates for every active Chrome Root Store anchor and every enterprise-added anchor. Derive certificate and SPKI SHA-256 identifiers.
- Generate CADistrustedCertificates from every anchor except the approved ca.scone.cloud key. Do not manually curate the base64 list.
- Deploy ca.scone.cloud with permitted_dns_names=["scone.cloud"], set CAPlatformIntegrationEnabled=false, CACertificateManagementAllowed=2 and QuicAllowed=false.
- Use URLBlocklist/URLAllowlist only as application hardening. Ensure policy distribution does not depend on a public-CA connection that the profile has just disabled.
- Restart Chrome and verify policy status, source and scope in chrome://policy. Verify the active root-store version again after restart.
- Run the approved-CA positive test, public-CA negative test, unrelated-name constraint test, public-site failure test and UDP/443/HTTP3 test.
- At every Chrome/root-store update, regenerate the policy and repeat the full negative suite before promotion.
- Fail closed for scone.cloud when root-store coverage, policy parsing, policy precedence or transport state cannot be proven.

MINIMAL FALLBACK POLICY SKELETON

```
{
 "CADistrustedCertificates": [
  "<GENERATED_BASE64_DER_FOR_EACH_NON_APPROVED_ANCHOR>"
 ],
 "CACertificatesWithConstraints": [
  {
   "certificate": "<BASE64_DER_CA_SCONE_CLOUD>",
   "constraints": {"permitted_dns_names": ["scone.cloud"]}
  }
 ],
 "CAPlatformIntegrationEnabled": false,
 "CACertificateManagementAllowed": 2,
 "QuicAllowed": false
}
```

Acceptance gate. Do not approve deployment merely because api.scone.cloud works with the private CA. The decisive test is that the same hostname with a valid alternate CA fails after every browser and root-store update.

9.11 Runbook for a mandatory TLS policy proxy

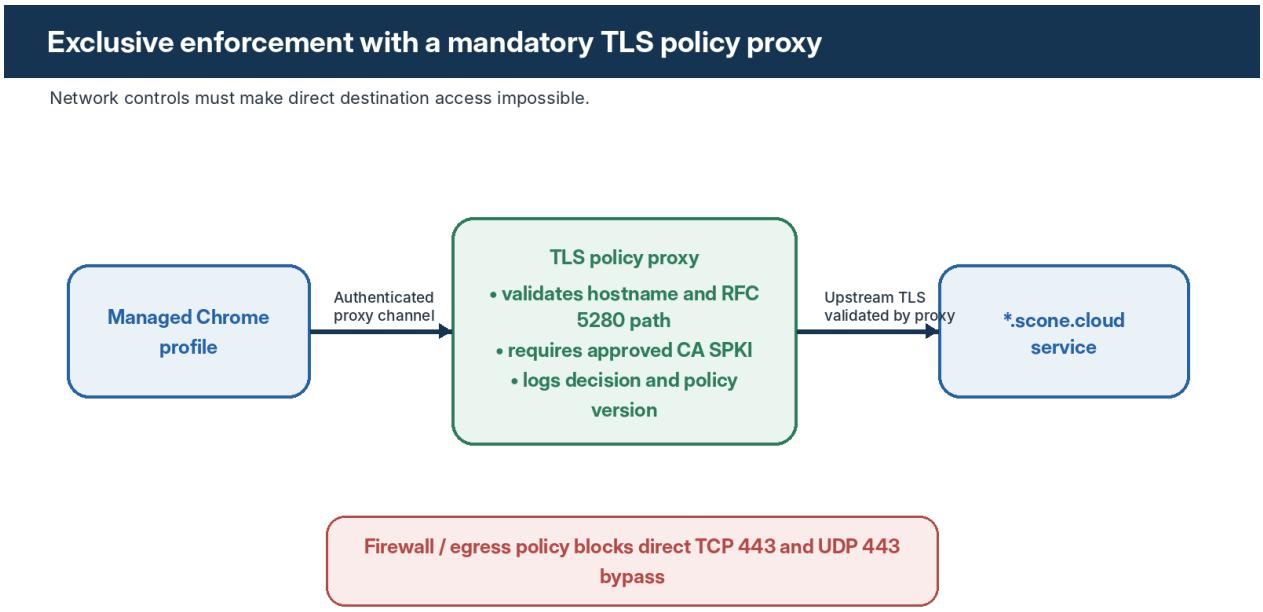


Figure 7 — Mandatory proxy architecture with explicit upstream CA validation and network anti-bypass controls.

- Configure a machine-level Chrome ProxySettings policy or mandatory PAC configuration. If a PAC file is used, enable ProxyPacMandatory so failure does not silently fall back to direct access. [18]

- At the operating-system or network layer, block direct TCP 443 and UDP 443 access from the managed browser endpoint to the protected service networks, except from the proxy.
- Authenticate the browser-to-proxy channel using an HTTPS proxy, device-bound credentials, mTLS or a controlled private network. Protect proxy policy and keys.
- In the proxy verifier, perform ordinary hostname and path validation first. For hostnames in scone.cloud, require an approved CA SPKI in the verified upstream chain.
- Disable or control QUIC so it cannot bypass the proxy. Test WebSocket CONNECT and long-lived reconnect behavior.
- Log hostname, selected chain fingerprints, approved-SPKI match, policy version and decision. Do not log application payloads unless separately authorized.
- Fail closed for the protected namespace on policy parse errors, proxy unavailability or upstream-verification uncertainty. Never fall back to direct.

PROXY PSEUDOCODE

```
function validate_upstream(hostname, peer_chain):
  result = ordinary_rfc5280_and_hostname_validation(hostname, peer_chain)
  if not result.ok:
    return DENY(result.error)

  if dns_subtree_match(hostname, "scone.cloud"):
    if not any(spki_sha256(cert) in APPROVED_CA_SPKIS
           for cert in result.verified_chain):
      return DENY("ERR_REQUIRED_CA_NOT_IN_VERIFIED_CHAIN")

  return ALLOW(result)
```

9.12 Runbook for custom Chromium enforcement

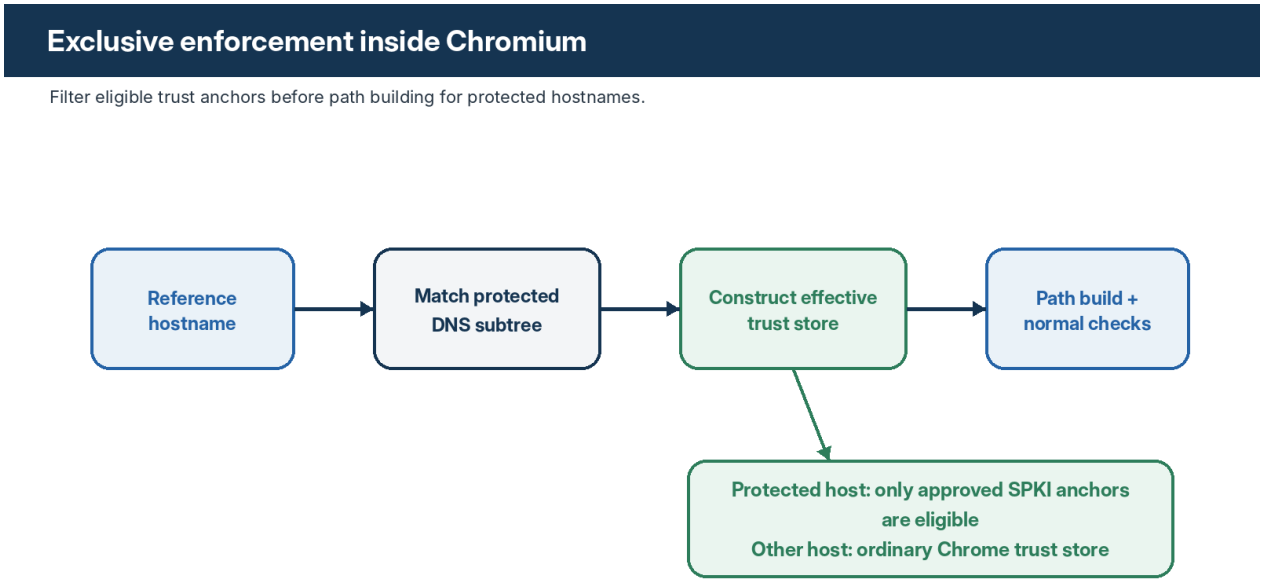


Figure 8 — Recommended custom-verifier design: filter eligible anchors before path construction.

- Add a signed machine policy that maps canonical DNS subtrees to one or more approved SPKI hashes.
- Parse the policy in the browser process alongside other certificate policies and send it to the cert-verifier service over the existing policy IPC path.
- Canonicalize the reference hostname to lowercase ASCII A-label form and perform label-aware subtree matching.
- Before path construction, filter the eligible trust anchors for a protected hostname to the approved SPKI set. Do not merely inspect the server-provided issuer string.
- Run ordinary path validation, hostname matching, revocation, algorithm and CT checks against the filtered trust store.
- Emit a dedicated NetLog event and error code when no approved path exists.
- Cover HTTP/1.1, HTTP/2, HTTP/3, WebSockets, service workers, preconnects, speculative requests and browser-internal requests in integration tests.
- Build reproducibly, sign packages and update on Chromium's security cadence.

9.13 CA rotation

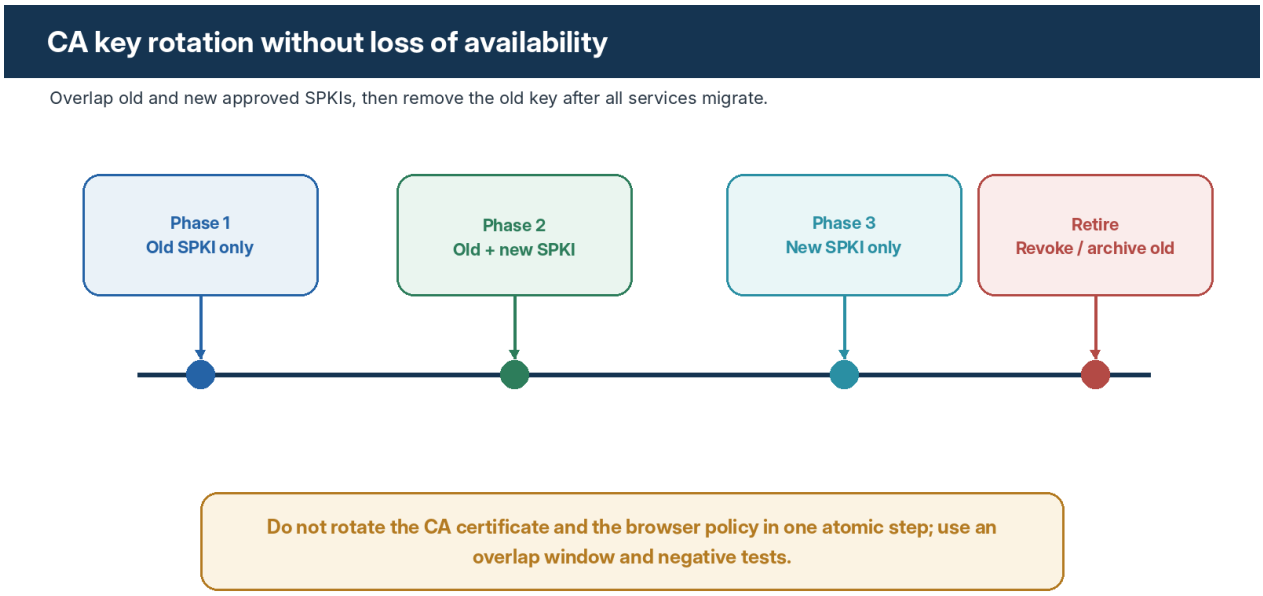


Figure 9 — Safe CA rotation uses an overlap window for old and new SPKIs.

- Generate the new CA key in the intended HSM and create the constrained CA certificate.
- Add the new SPKI to the exclusive allowlist while retaining the old SPKI.
- Deploy the new CA certificate through Chrome policy before any service begins serving new-chain certificates.
- Migrate a canary service, run the complete positive and negative test suite, then migrate the fleet.
- Wait until all sessions and cached chains using the old CA have expired or reconnected.
- Remove the old SPKI from the required-anchor policy and remove the old CA trust entry.
- Revoke or archive the old CA according to incident and retention policy; keep evidence of the transition.

9.14 Incident response

Table 12 — Incident-response actions

Trigger	Immediate action	Evidence
Unexpected public certificate observed for scone.cloud	Block affected hostname or entire subtree at the exclusive enforcement layer; determine whether it was accepted.	Leaf and chain DER, SCTs, DNS, NetLog, CT entry, policy version.
Approved CA key suspected compromised	Add its SPKI to global distrust, deploy replacement overlap only if risk permits, revoke affected leaves.	HSM audit, issuance log, certificate inventory.
Policy missing or invalid	Fail closed for protected namespace, restore signed known-good policy, investigate source precedence.	chrome://policy export, MDM/GPO audit, file/registry hashes.
Proxy bypass detected	Block direct egress, isolate endpoint, review QUIC and alternate network paths.	Firewall, proxy, DNS and endpoint telemetry.
Custom browser out of security support	Disable protected service access until patched build is deployed.	Browser version, build provenance and update logs.
Active root store contains an SPKI absent from the global-distrust policy	Block protected-service access for the fallback profile; regenerate and redeploy policy; rerun alternate-CA test.	chrome://system root-store version, root manifest, generated policy hash, chrome://policy export, negative-test result.

- Preserve evidence before clearing caches or restarting services where feasible.
- Classify the event as misissuance, interception, CA compromise, DNS/routing attack, server misconfiguration or policy bypass.
- Run the alternate-CA negative test after remediation before reopening access.
- Document whether the event was prevented, detected after acceptance or merely observed in CT.

9.15 Rollback and break-glass

Rollback must not silently weaken the protected namespace. Maintain a signed previous browser build and policy bundle that implements the same exclusive rule. A break-glass policy should be time-limited, require two-person approval, name the exact temporary CA SPKI and hostname scope, and generate a high-severity alert. Do not implement break-glass by enabling all platform roots or by using ignore-certificate-errors flags.

9.16 Troubleshooting

Table 13 — Troubleshooting guide

Symptom	Likely cause	Corrective action
Policy absent in chrome://policy	Wrong path/domain, unmanaged profile, policy precedence or invalid deployment.	Check policy source, ownership, JSON/plist/registry syntax and management enrollment.
Policy shows error	PEM text instead of base64 DER, whitespace/invalid base64, missing constraints, invalid DNS value.	Recreate DER/base64 and validate policy JSON.
Approved service fails	Wrong SAN, disallowed extra SAN, missing intermediate, expired CA, revocation responder unavailable.	Inspect server chain and CA constraints; capture NetLog.
Unrelated domain works through private CA	Constraint not loaded, CA trusted elsewhere without constraint, or another path selected.	Disable alternate trust entry and verify actual path.
Public certificate still works	Expected stock-Chrome behavior.	Deploy the exclusive proxy or custom verifier.
Proxy policy appears set but direct access works	PAC fallback, alternate network interface, QUIC or firewall gap.	Enable mandatory PAC behavior and enforce egress at OS/network level.
Custom verifier rejects despite approved chain existing	Path builder selected another anchor before post-check.	Filter eligible anchors before path building or retry with restricted trust store.
Private CA works over TCP but HTTP/3 fails or behaves differently	QUIC does not use locally added trust to add a private anchor.	Set QuicAllowed=false, restart Chrome and verify TCP fallback/absence of UDP 443 in NetLog.
Public certificate unexpectedly works in the global-distrust profile	New root-store anchor, incomplete generated list, other enterprise trust source or policy precedence failure.	Compare active SPKIs with policy, inspect chrome://policy source/scope, block access until coverage is complete.

10. Security analysis

10.1 Security properties by architecture

Table 14 — Security-property comparison

Property	Stock constrained trust	Mandatory proxy	Custom Chromium
Private CA cannot issue outside scone.cloud	Yes, when constraints are correctly applied.	Yes, if proxy also validates constraint.	Yes.
Public CA cannot authenticate scone.cloud	No.	Yes, if proxy is non-bypassable.	Yes.
Platform interception CA cannot authenticate scone.cloud	Partially with platform integration disabled.	Yes, on upstream side; browser-proxy trust remains separate.	Yes, unless explicitly approved.
Works for HTTP/3 and WebSockets	WebSockets use normal Chrome validation. Locally added positive trust is not considered for QUIC; disable QUIC or rely on explicit TCP fallback.	Only if proxy and egress design covers or disables QUIC; WebSockets must traverse the proxy.	Yes when the custom check is integrated into every relevant verifier/transport path.
Central policy/logging	Yes.	Strong.	Strong with enterprise diagnostics.
Browser update burden	Normal Chrome.	Normal Chrome plus proxy operations.	Custom browser security maintenance.

10.1.1 Conditional guarantee of the global-distrust fallback

The fallback provides the desired result only under a conjunction of operational assumptions. It should be documented as a conditional guarantee, not as an intrinsic Chrome domain policy:

- G1 — every active public and enterprise anchor other than ca.scone.cloud is represented by a distrusted SPKI;
- G2 — platform integration is disabled and no higher-precedence policy adds another eligible anchor;
- G3 — the generated policy is accepted without truncation, schema error or delivery failure;
- G4 — QUIC is disabled, so locally added positive trust is evaluated on TCP TLS;
- G5 — ca.scone.cloud remains constrained to scone.cloud and ordinary certificate validation remains successful; and
- G6 — browser/root-store updates are blocked from production until G1–G5 are re-established.

If any assumption fails, the organization cannot claim exclusive binding. A native RequiredTrustAnchors rule is materially stronger because the protected hostname itself selects the eligible anchor set; it does not depend on reconstructing the complement of a moving public root store.

10.2 Attack scenarios

Table 15 — Threat analysis

Scenario	Baseline result	Exclusive architecture result	Residual risk
Rogue public CA issues api.scone.cloud	Chrome may accept; CT can reveal it.	Rejected by proxy/custom verifier.	Endpoint policy or exclusive component compromise.
Enterprise root installed in OS	Blocked if platform integration disabled and no other path.	Rejected upstream or filtered in verifier.	Machine administrator can change policy.
Attacker controls DNS and route	TLS still requires a trusted path.	Requires approved CA path; attack fails without it.	Compromise of approved CA or endpoint.

Scenario	Baseline result	Exclusive architecture result	Residual risk
Approved CA key compromised	Attacker can issue protected certificates.	Still accepted because key is approved.	HSM, short lifetimes, revocation and rapid policy rotation are essential.
Server serves mixed-SAN certificate	Constrained path should fail if any DNS SAN is outside subtree.	Same, plus required-anchor check.	Client interoperability and issuance validation.
Cross-signed CA provides alternate root	Stock path builder may select public root.	Anchor filtering / proxy SPKI check rejects unapproved root path.	Policy must define whether approved intermediate anywhere in chain is sufficient or only approved anchor.
Chrome Root Store adds a new public root after the distrust snapshot	The new root is eligible until policy coverage catches up.	Native host-scoped filtering is unaffected; fallback deployment must fail its reconciliation gate.	Update pipeline or policy-delivery compromise.

10.3 Trust-boundary choices

The custom-browser architecture trusts the browser binary, its updater, the policy distribution channel and the endpoint operating system. The proxy architecture additionally trusts the proxy, its upstream verifier, network anti-bypass controls and the browser-to-proxy authentication. In exchange, it avoids maintaining a browser fork. The correct choice depends on which component can be controlled, attested and audited most strongly.

10.4 SPKI matching semantics

A policy must define what it means for a chain to "contain the approved CA." Requiring the final trust anchor SPKI is simplest and prevents cross-signing from changing the trust root. Requiring an approved intermediate anywhere in the chain permits a dedicated issuing CA to remain approved even when anchored differently, but may unintentionally accept a public cross-sign. The recommended default is to approve trust-anchor SPKIs and, where necessary, a separately enumerated issuing-CA SPKI with explicit chain-position semantics.

10.5 Fail-closed requirements

- Policy parse failure for a protected namespace must deny, not revert to ordinary Web PKI.
- Unknown or non-canonical hostnames must not bypass label-aware matching.
- A proxy outage must not trigger direct fallback.
- A custom-verifier internal error must return a certificate error for protected hosts.
- A CA rotation policy must retain at least one valid approved path throughout the overlap, then remove the old key deliberately.
- Observability failures should alert but should not cause a verifier to accept an otherwise disallowed path.

11. Chromium enhancement proposal: RequiredTrustAnchors

The missing primitive can be implemented as a new enterprise policy that selects eligible trust anchors by reference hostname. The working name in this whitepaper is RequiredTrustAnchors. It is a proposal, not a current Chrome policy.

11.1 Proposed schema

PROPOSED JSON

```
{
  "RequiredTrustAnchors": [
    {
      "dns_names": ["scone.cloud"],
      "required_anchor_spki_sha256": [
        "BASE64_SHA256_SPKI_OLD",
        "BASE64_SHA256_SPKI_NEW"
      ],
      "required_intermediate_spki_sha256": [],
      "mode": "fail_closed"
    }
  ]
}
```

Table 16 — Proposed policy semantics

Field	Semantics
dns_names	Label-aware permitted subtrees in canonical A-label form. A value scone.cloud matches the apex and descendants.
required_anchor_spki_sha256	Eligible final trust-anchor keys for matching hostnames.
required_intermediate_spki_sha256	Optional explicitly approved issuing intermediates; empty by default.
mode	fail_closed is the production mode. A report_only mode may support migration but must never be confused with enforcement.

11.2 Recommended verifier algorithm

PSEUDOCODE

```
function VerifyServerCertificate(hostname, presented_chain, policy):
  h = CanonicalizeDnsReferenceName(hostname)
  rule = MostSpecificRequiredTrustAnchorRule(h, policy)

  if rule is None:
    return OrdinaryChromeVerify(h, presented_chain)

  effective_trust_store = FilterTrustAnchors(
    ordinary_trust_store,
    anchor -> SpkiSha256(anchor) in rule.required_anchor_spki_sha256)

  result = OrdinaryChromeVerifyWithTrustStore(
    h, presented_chain, effective_trust_store)

  if not result.ok:
    return result

  if rule.required_intermediate_spki_sha256 is not empty and
    not ChainContainsOneOf(result.verified_chain,
                           rule.required_intermediate_spki_sha256):
    return ERR_REQUIRED_CA_NOT_IN_VERIFIED_CHAIN

  return result
```

Filtering anchors before path building is preferable to a simple post-validation check. A post-check can reject a connection when the verifier happened to choose an unapproved path even though an approved alternative path

exists. It then needs a second restricted path-building pass. Pre-filtering makes the policy a property of the effective trust store and produces deterministic path selection.

11.3 Chromium integration points

- Define the policy schema in Chromium's enterprise policy templates with dynamic refresh and per-profile semantics matching other certificate policies.
- Parse and validate DNS subtree strings and SPKI hashes in the browser process.
- Extend the certificate-verifier service IPC structure with required-anchor rules.
- Pass the canonical reference hostname into trust-store selection before CertVerifyProcBuiltin path construction.
- Preserve CRLSet, online revocation, Name Constraints, algorithm and CT processing after anchor filtering.
- Emit NetLog events containing the matched rule identifier, eligible anchors and final decision without exposing secret material.
- Expose the effective policy in chrome://policy and a certificate diagnostics view.

11.4 Policy matching rules

- Match DNS labels, not raw suffix strings: evil-scone.cloud must not match scone.cloud.
- Canonicalize IDNs to A-label form and lowercase before matching.
- Use the most specific matching subtree when multiple rules apply.
- Treat IP literals separately; do not apply DNS suffix rules to textual IP addresses.
- Reject malformed policy entries for their protected scope, and surface a high-severity diagnostic.
- Do not permit web content, extensions or server headers to weaken the machine policy.

11.5 Test requirements

Table 17 — Minimum conformance suite for a RequiredTrustAnchors implementation

Test family	Required cases
Path building	Approved root, unapproved root, cross-signed intermediate, AIA alternate path, cached alternate path.
DNS matching	Apex, one-level and multi-level subdomains, suffix confusion, uppercase, trailing dot, IDN A-label.
Transports	HTTP/1.1, HTTP/2, HTTP/3, WebSocket initial and reconnect, service-worker fetch, preconnect.
Policy lifecycle	Dynamic refresh, malformed policy, source precedence, profile isolation, incognito behavior.
Rotation	Old only, old+new overlap, new only, rollback.
Diagnostics	Dedicated error, NetLog event, chrome://policy status and report-only telemetry.

12. Applying the design to confidential-computing services

Confidential computing protects service execution from a cloud or infrastructure operator. The browser still needs a conventional, low-latency way to authenticate the service endpoint. A private CA can bridge remote attestation and standard TLS: the CA issues a short-lived server certificate only after the workload presents acceptable attestation evidence and satisfies a deployment policy.

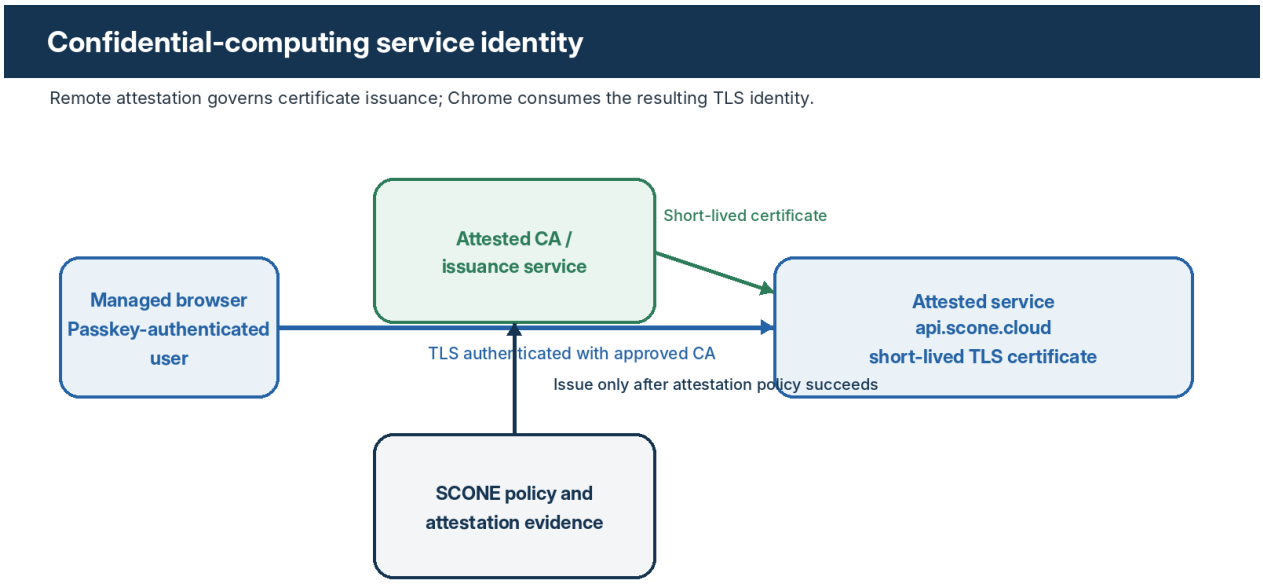


Figure 10 — Attestation-gated certificate issuance for a SCONE service.

12.1 Division of responsibilities

Table 18 — Complementary controls in the confidential-service architecture

Control	Question answered
TLS server certificate	Which DNS service endpoint and public key is the browser communicating with?
Exclusive CA binding	Was that endpoint identity authorized by the designated attestation-aware CA rather than any other trusted CA?
Remote attestation	Is the service running the approved code and configuration in the expected confidential-computing environment?
Passkey / WebAuthn	Which user or authenticator is accessing the service, and is the assertion bound to the web origin?
SCONE policy / control plane	Which measurements, configurations, secrets and update policies are allowed?

12.2 Attestation-gated issuance flow

- The workload generates or receives a service key inside the confidential execution boundary.
- The workload submits a CSR and fresh attestation evidence to the issuance service.
- The issuance service validates hardware evidence, workload measurement, configuration, freshness and SCONE policy.
- On success, the dedicated CA issues a short-lived certificate whose DNS SAN is within scone.cloud.
- The browser validates the certificate and, with exclusive binding, requires the approved CA key.

- Renewal repeats attestation, so continued service identity is coupled to approved execution state.

This design does not make the browser a general attestation verifier. It converts a successful attestation decision into a standard TLS credential that every browser already understands. The exclusive CA rule is important because otherwise a separate public CA could issue the same DNS identity without attesting the workload, bypassing the intended attestation-to-identity binding.

12.3 CA and key-management recommendations

- Keep the root CA offline and use a dedicated pathLen=0 issuing CA for the protected namespace.
- Keep issuing keys in an HSM or attested signing service with multi-party administrative controls.
- Issue short-lived leaf certificates and automate renewal after fresh attestation.
- Bind issuance authorization to exact DNS names, tenant identifiers and workload policy, not only to possession of a CSR key.
- Maintain an append-only issuance log containing certificate fingerprint, SANs, attestation result, policy version and validity period.
- Separate browser policy rotation from CA key rotation using the overlap procedure in Chapter 9.

12.4 User authentication remains separate

Passkeys bind a user assertion to the relying-party origin. They do not independently prove that the TLS certificate was issued by the intended private CA. Conversely, exclusive CA binding authenticates the service but not the user. A secure confidential service should use both: strict server identity before WebAuthn is invoked, then passkey authentication for the user session.

13. Conclusions and recommendations

Chrome Constrained Certificate Trust is a meaningful enterprise security feature. It lets an administrator add a CA while restricting that CA to a controlled DNS or IP namespace. A CA certificate with a critical Name Constraints extension can provide the same directional restriction in the certificate itself, and current Chromium enterprise anchor plumbing enforces embedded anchor constraints for CACertificates. These controls answer “where may this CA be used?”

They do not answer the inverse question “which CA must be used here?” Stock Chrome continues to accept a valid alternative path to another eligible trust anchor. Therefore, the exact domain-scoped requirement that only ca.scone.cloud authenticate *.scone.cloud cannot be guaranteed through CACertificatesWithConstraints, embedded Name Constraints, CAA, CT, platform-integration controls or an extension alone.

> Final recommendation For a managed confidential-computing deployment, use a dedicated name-constrained issuing CA, deploy Chrome’s constrained-trust baseline, disable unnecessary platform anchors and QUIC for locally added private trust, prevent user CA changes, publish CAA and monitor issuance. Add custom Chromium trust-anchor filtering for the strongest browser-local guarantee, or a mandatory non-bypassable TLS policy proxy for the most practical enterprise rollout. Use complete global distrust only for a controlled single-purpose fallback with continuous root-store reconciliation.

A dedicated single-purpose profile can approximate the property for TCP TLS by globally distrusting every other active anchor, disabling platform integration and QUIC, and continuously reconciling the root-store snapshot. This is a qualified operational workaround, not a domain-scoped policy: public Web access fails, a new root creates a coverage gap until policy is updated, and the same distrusted SPKI cannot be re-added with narrower constraints. The general managed-browser recommendation therefore remains a custom Chromium required-anchor rule or a mandatory non-bypassable TLS policy proxy.

13.1 Minimum production checklist

- Protected subtree is scone.cloud or narrower, never the whole public .cloud suffix.
- CA certificate and SPKI fingerprints are approved out of band.
- CA carries critical Name Constraints or Chrome supplies external constraints.
- chrome://policy shows mandatory, error-free certificate policies.
- Platform CA import and user CA management are disabled unless explicitly required.
- If the single-purpose global-distrust fallback is used, every active non-approved anchor SPKI is covered and the update reconciliation gate is operational.
- An exclusive enforcement point is deployed and fail-closed.
- Direct proxy bypass is tested and blocked where a proxy is used; QUIC is disabled or explicitly covered by the selected enforcement architecture.
- Positive, alternate-CA, cross-sign, WebSocket, service-worker and transport-specific tests pass; a local-private-CA stock profile does not rely on QUIC.
- CAA, public CT monitoring and private issuance audit are operational.
- CA rotation and incident rollback have been rehearsed.

Appendix A. Complete Linux baseline policy

/ETC/OPT/CHROME/POLICIES/MANAGED/SCONE.JSON

```
{
 "CACertificatesWithConstraints": [
  {
   "certificate": "BASE64_DER_CA",
   "constraints": {
    "permitted_dns_names": [
     "scone.cloud"
    ]
   }
  }
 ],
 "CAPlatformIntegrationEnabled": false,
 "CACertificateManagementAllowed": 2,
 "RequireOnlineRevocationChecksForLocalAnchors": true,
 "URLBlocklist": ["*"],
 "URLAllowlist": [
  "https://scone.cloud",
  "https://[*.]scone.cloud",
  "https://pki.scone.cloud"
 ]
}
```

Validate URL filter syntax against the current Chrome administrator documentation and add every required identity-provider, update, OCSP and CRL endpoint. URL filtering does not create exclusive CA binding.

Appendix B. Embedded Name Constraints checklist

- Extension appears only in a CA certificate and is marked critical.
- permittedSubtrees contains dNSName scone.cloud.
- basicConstraints is critical with CA:TRUE and an appropriate pathLen.
- keyUsage is critical and contains keyCertSign and cRLSign.
- Leaf issuance rejects any DNS SAN outside the intended subtree.
- CSR copy_extensions behavior cannot smuggle unapproved SANs into a certificate.
- Chrome and every other required client have been tested with the exact chain.

Appendix C. Exclusive-enforcement test matrix

Table 19 — Acceptance and bypass tests

ID	Scenario	Expected result
T01	Approved CA, apex scone.cloud	Pass
T02	Approved CA, api.scone.cloud	Pass
T03	Approved CA, deep.api.scone.cloud	Pass
T04	Approved CA, unrelated.example	Fail
T05	Approved CA, mixed allowed/disallowed DNS SANs	Fail
T06	Public CA, api.scone.cloud	Fail under exclusive layer
T07	Platform interception CA, api.scone.cloud	Fail
T08	Cross-signed approved issuing CA to unapproved root	Result matches explicit chain-position policy
T09	Expired approved leaf	Fail
T10	Revoked approved leaf	Fail according to revocation policy
T11	Unreachable OCSP with hard-fail enabled	Fail
T12	Wrong hostname	Fail
T13	HTTP/2	Same CA decision
T14	HTTP/3 with stock locally added private CA	QUIC disabled or connection falls back to TCP; do not claim local positive trust over QUIC
T15	WebSocket initial connection	Same CA decision
T16	WebSocket reconnect after certificate swap	New connection fails
T17	Service-worker background fetch	Same CA decision
T18	PAC unavailable	No direct fallback
T19	Direct TCP 443 attempt around proxy	Blocked
T20	Direct UDP 443 attempt around proxy	Blocked
T21	Old CA only during phase 1	Pass
T22	Old and new during overlap	Both pass
T23	Old after retirement	Fail
T24	Malformed required-anchor policy	Fail closed for protected namespace
T25	Chrome Root Store adds an anchor after global-distrust snapshot	Coverage gate fails; protected access remains blocked until policy is regenerated

ID	Scenario	Expected result
T26	Same public-root SPKI in trust and distrust policies	Fail; distrust overrides trust
T27	Ordinary public website in dedicated global-distrust profile	Fail by design
T28	Valid public-CA certificate for api.scone.cloud after complete global distrust	Fail over TCP
T29	QUIC disabled policy and UDP/443 observation	No successful QUIC connection; TCP path is used

Appendix D. Evidence collection commands

SHELL

```
# Effective Chrome policies
# Open chrome://policy -> Reload policies -> Export policies

# Chrome NetLog
# Open chrome://net-export -> Start Logging to Disk

# Server chain
openssl s_client -connect api.scone.cloud:443 \
  -servername api.scone.cloud -showcerts </dev/null

# Certificate fingerprint
openssl x509 -in leaf.pem -noout -fingerprint -sha256

# SPKI hash
openssl x509 -in leaf.pem -pubkey -noout | \
  openssl pkey -pubin -outform DER | \
  openssl dgst -sha256 -binary | openssl base64 -A

# DNS CAA
dig +short CAA scone.cloud

# DNS resolution evidence
dig +dnssec api.scone.cloud A AAAA
```

Appendix E. Reference policy proposal

PROPOSED CHROME ENTERPRISE POLICY

```
{
  "RequiredTrustAnchors": [
    {
      "dns_names": ["scone.cloud"],
      "required_anchor_spki_sha256": [
        "BASE64_SHA256_SPKI_CURRENT",
        "BASE64_SHA256_SPKI_NEXT"
      ],
      "mode": "fail_closed"
    }
  ]
}
```

This is not a currently supported Chrome policy. It is included as a concrete design target for a Chromium patch or upstream feature proposal.

Appendix F. Dedicated-browser root-store reconciliation

This appendix provides an implementation pattern for generating the full-root distrust policy. It assumes that the operator has obtained an exact, reviewed directory of DER or PEM certificates representing the active Chrome Root Store and any additional enterprise anchors. The policy generator excludes the approved CA by SPKI and emits base64 DER entries accepted by CADistrustedCertificates.

F.1 Inputs and evidence

- Pinned Chrome binary version and active chrome_root_store component version.
- Exact root certificate corpus for that version, with source revision and hash.
- Approved ca.scone.cloud DER certificate and approved SPKI SHA-256 value.
- Inventory of all enterprise-added trust anchors and policy sources.
- Policy output hash, policy deployment evidence and alternate-CA negative-test evidence.

F.2 Example policy generator

PYTHON 3 — REQUIRES THE cryptography PACKAGE

```
#!/usr/bin/env python3
import argparse, base64, hashlib, json
from pathlib import Path
from cryptography import x509
from cryptography.hazmat.primitives import serialization


def load_cert(path: Path) -> x509.Certificate:
  data = path.read_bytes()
  try:
    return x509.load_pem_x509_certificate(data)
  except ValueError:
    return x509.load_der_x509_certificate(data)


def spki_sha256(cert: x509.Certificate) -> str:
  spki = cert.public_key().public_bytes(
    serialization.Encoding.DER,
    serialization.PublicFormat.SubjectPublicKeyInfo,
  )
  return hashlib.sha256(spki).hexdigest()


ap = argparse.ArgumentParser()
ap.add_argument('--roots', type=Path, required=True)
ap.add_argument('--approved-ca', type=Path, required=True)
ap.add_argument('--out', type=Path, required=True)
args = ap.parse_args()

approved = load_cert(args.approved_ca)
approved_spki = spki_sha256(approved)
entries = []
manifest = []

for path in sorted(p for p in args.roots.iterdir() if p.is_file()):
  cert = load_cert(path)
  der = cert.public_bytes(serialization.Encoding.DER)
  spki = spki_sha256(cert)
  manifest.append({
    'file': path.name,
    'certificate_sha256': hashlib.sha256(der).hexdigest(),
    'spki_sha256': spki,
    'subject': cert.subject.rfc4514_string(),
  })
  if spki != approved_spki:
    entries.append(base64.b64encode(der).decode('ascii'))

if not entries:
  raise SystemExit('refusing to emit an empty distrust list')

output = {
  'CADistrustedCertificates': entries,
  '_evidence': {
    'approved_ca_spki_sha256': approved_spki,
    'root_count': len(manifest),
    'distrusted_count': len(entries),
    'manifest': manifest,
```

```
    },
  }
  args.out.write_text(json.dumps(output, indent=2) + '\n')
  print(f'wrote {len(entries)} distrust entries to {args.out}')
```

Remove the _evidence object before deploying the value if the policy channel requires only recognized Chrome policy keys. Retain the evidence manifest separately and sign or hash it. The script intentionally works from certificate files because CADistrustedCertificates accepts base64 DER certificates and derives distrust from their SPKIs.

F.3 Promotion gate

- The source root-store version equals the active version shown by the target browser.
- The generated inventory contains every active public and enterprise anchor.
- The approved CA SPKI is absent from the distrust entries and present exactly once in the constrained-trust policy.
- Policy size and transport through GPO/MDM/cloud management are verified without truncation.
- chrome://policy reports all certificate and QUIC policies as mandatory and error-free.
- api.scone.cloud succeeds with the approved CA and fails with a valid alternate CA.
- An ordinary public HTTPS site fails in the dedicated profile, confirming that public roots are not accidentally eligible.
- NetLog and endpoint telemetry show no successful QUIC/UDP 443 connection.

Rollback rule. A rollback may restore a previously approved Chrome/root-store/policy tuple. Do not roll back only the distrust policy while leaving a newer root-store component active, because the coverage invariant may no longer hold.

References

[1] RFC 5280 — Internet X.509 Public Key Infrastructure Certificate and CRL Profile. https://www.rfc-editor.org/rfc/rfc5280.html Name Constraints and certification path validation.

[2] Chromium: Certificate verification architecture. https://chromium.googlesource.com/chromium/src/+/HEAD/net/cert/README.md

[3] Chromium: net/cert/cert_verify_proc.h. https://chromium.googlesource.com/chromium/src/+/refs/heads/main/net/cert/cert_verify_proc.h Instance parameters for anchors, constraints and distrust.

[4] Chromium: profile_network_context_service.cc. https://chromium.googlesource.com/chromium/src/+/refs/heads/main/chrome/browser/net/profile_network_context_service.cc Enterprise certificate policy processing.

[5] Chromium policy definition: CACertificatesWithConstraints. https://chromium.googlesource.com/chromium/src/+/refs/heads/main/components/policy/resources/templates/policy_definitions/CertificateManagement/CACertificatesWithConstraints.yaml

[6] Chrome Enterprise policy: CACertificates. https://chromeenterprise.google/policies/ca-certificates/

[7] Chrome Enterprise policy: CADistrustedCertificates. https://chromeenterprise.google/policies/ca-distrusted-certificates/

[8] Chrome Enterprise policy: CAPlatformIntegrationEnabled. https://chromeenterprise.google/policies/ca-platform-integration-enabled/

[9] Chrome Enterprise policy: CACertificateManagementAllowed. https://chromeenterprise.google/policies/ca-certificate-management-allowed/

[10] Chrome Enterprise policy: RequireOnlineRevocationChecksForLocalAnchors. https://chromeenterprise.google/policies/require-online-revocation-checks-for-local-anchors/

[11] Chromium Chrome Root Store FAQ. https://chromium.googlesource.com/chromium/src/+/main/net/data/ssl/chrome_root_store/faq.md

[12] Chrome Enterprise policy list. https://chromeenterprise.google/policies/

[13] RFC 8659 — DNS Certification Authority Authorization (CAA). https://www.rfc-editor.org/rfc/rfc8659.html

[14] RFC 9162 — Certificate Transparency Version 2.0. https://www.rfc-editor.org/rfc/rfc9162.html

[15] Chromium: Certificate Transparency. https://chromium.googlesource.com/chromium/src/net/+/HEAD/docs/certificate-transparency.md

[16] Chrome Extensions: chrome.webRequest API. https://developer.chrome.com/docs/extensions/reference/api/webRequest

[17] Chromium intent: deprecate and remove public key pinning. https://groups.google.com/a/chromium.org/g/blink-dev/c/he9tr7p3rZ8

[18] Chrome Enterprise URLAllowlist and proxy policies. https://chromeenterprise.google/policies/url-allowlist/ Use with current ProxySettings documentation.

[19] Chrome Enterprise policy management for Windows. https://support.google.com/chrome/a/answer/187202

[20] Chrome Enterprise policy management for Linux. https://support.google.com/chrome/a/answer/7517525

[21] Chrome Enterprise policy management for macOS. https://support.google.com/chrome/a/answer/7517624

[22] Mozilla Firefox enterprise policy reference. https://mozilla.github.io/policy-templates/ For comparative certificate-installation capabilities.

[23] Chrome Enterprise policy: ChromeRootStoreEnabled (retired transition policy). https://chromeenterprise.google/policies/chrome-root-store-enabled/ See also the Chromium Chrome Root Store FAQ history.

[24] Chromium: cert_verifier_policy_browsertest.cc — TestDistrustOverridesTrust. https://chromium.googlesource.com/chromium/src/+/main/chrome/browser/net/cert_verifier_policy_browsertest.cc

[25] Chromium: root_store.proto. https://chromium.googlesource.com/chromium/src/+/main/net/cert/root_store.proto Current metadata ConstraintSet exposes permitted_dns_names.

[26] Chrome Enterprise policy: QuicAllowed. https://chromeenterprise.google/policies/quic-allowed/

[27] Chromium: current Chrome Root Store manifest and textproto. https://chromium.googlesource.com/chromium/src/+/main/net/data/ssl/chrome_root_store/root_store.md and root_store.textproto

All web references were verified on 26 July 2026. Version 2.1 additionally verified Chrome Root Store update behavior, QUIC local-trust semantics, external constraint schema and Chromium distrust precedence. Where source and product documentation differ, the deployed browser version and its chrome://policy, chrome://system and NetLog diagnostics are authoritative for operations.